Python + SQLite (sqlite3): Complete Teaching Notes

Nov 4, 2026 · @kamos

Course overview

By the end of this course, a student can design a small database, build it from Python with the built-in `sqlite3` module, and write programs that add, search, change, report on and back up real data. You said the name slightly differently: the database is called **SQLite** (say "S-Q-L-ite") and the Python module is spelled **`sqlite3`**. There is nothing to install, because it ships with Python.

Who this is for: students who know Python basics (variables, `if`, loops, functions, lists, dictionaries). No database knowledge is needed.

What students will be able to do after the course

- Explain what a database, table, row, column, primary key and foreign key are.
- Connect to a SQLite database from Python and run SQL safely.
- Do all four CRUD operations: Create, Read, Update, Delete.
- Link tables together and combine them with JOINs.
- Group and summarise data (totals, averages, counts) for reports.
- Protect data with transactions, constraints and parameterised queries.
- Build two complete mini-systems: a school management system and a coffee farm harvest tracker.

Running examples used throughout. Every concept is taught with a real-life story, so students see why it matters:

Example	Where it appears	Why it works for teaching
School / academy (students, courses, enrolments, fees in UGX)	Most lessons and Part 10	Everyone understands registration and fee payment
Shop (products, stock, sales)	Quick examples	Shows stock updates and totals
Coffee farm (plots, harvests, kilos, buyers)	Part 11	Shows reports, dates and money in an agricultural setting

How to use these notes when teaching: each Part has (1) the idea in plain words, (2) a real-life story, (3) code to type live, (4) what the output looks like, and (5) common mistakes. Ask students to type every example themselves, not copy and paste. Part 13 has exercises with solutions, and Part 14 gives a lesson-by-lesson plan and a one-page cheat sheet.

Part 1: Databases, SQL and SQLite

1.1 What is a database?

A database is an organised place to store data so you can find, change and protect it quickly. **Real-life story:** a school keeps student records in an exercise book. Finding "all students who owe fees" means reading every page. In a database, that is one question that returns an answer in milliseconds, and the data cannot be torn, lost or written twice by accident.

1.2 Key vocabulary

Term	Meaning	School example
Database	A collection of related tables, stored in one file in SQLite	`school.db`
Table	A grid of data about one kind of thing	`students`
Column (field)	One property of that thing, with a type	`name`, `age`, `email`
Row (record)	One single item	One student: Aisha, 17, aisha@mail.com
Primary key	A column whose value is unique for every row, used as its ID	`student_id`
Foreign key	A column that points to the primary key of another table	`enrolments.student_id`
Query	A question you ask the database in SQL	"Which students are in Python?"
Schema	The design: table names, columns, types and rules	The blueprint of `school.db`

1.3 What is SQL?

SQL (Structured Query Language) is the language for talking to databases. It reads almost like English. The main groups of commands:

Group	Purpose	Commands
DDL (Data Definition)	Build and change the structure	`CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`
DML (Data Manipulation)	Change the data	`INSERT`, `UPDATE`, `DELETE`
DQL (Data Query)	Read the data	`SELECT`
TCL (Transaction Control)	Save or undo a group of changes	`COMMIT`, `ROLLBACK`

1.4 What is SQLite?

SQLite is a complete SQL database engine in a single small library. There is **no server to install, no password to set up and no network**. The whole database is one ordinary file (for example `school.db`) that you can copy, email or back up like a photo.

Why it is perfect for teaching and for many real apps:

- Zero setup: Python already includes it, so students start in the first minute.
- One file: easy to share homework and to back up.
- Real SQL: what students learn transfers to MySQL, PostgreSQL and SQL Server.
- Used everywhere: phones (Android and iOS apps), web browsers, desktop apps, small websites and farm or shop tools that must work offline.

1.5 SQLite compared with bigger databases

Feature	SQLite	MySQL / PostgreSQL
Needs a server running	No	Yes
Storage	One file	Server-managed files
Setup	None	Install, configure, create users
Many people writing at the same time	Limited (one writer at a time)	Excellent

Feature	SQLite	MySQL / PostgreSQL
Best for	Learning, apps, prototypes, offline and small to medium systems	Large multi-user web systems

Teaching point: tell students that SQLite is not "a toy". It is the most widely deployed database in the world. The honest limit is many users writing at once, which is when a company moves to PostgreSQL or MySQL.

1.6 What is the `sqlite3` module?

`sqlite3` is Python's built-in library (part of the standard library, like `math` or `random`) that lets Python programs talk to a SQLite database. You write SQL as text inside Python strings, and the module sends it to SQLite and brings the results back as Python tuples or rows.

Check it works and see versions:

```
import sqlite3

print(sqlite3.sqlite_version)   # version of the SQLite engine, e.g. 3.45.1
print(sqlite3.sqlite_version_info)   # the same version as a tuple, e.g. (3,
45, 1)
```

If `import sqlite3` runs without an error, you are ready.

Part 2: Connecting, cursors, commit and close

2.1 The five-step workflow (teach this first)

Every database program in Python follows the same steps. Students should memorise them.

1. **Connect**: open (or create) the database file.
2. **Cursor**: get a "pen" that sends SQL commands.
3. **Execute**: run SQL with the cursor.
4. **Commit**: save any changes permanently (needed for INSERT, UPDATE, DELETE, CREATE).
5. **Close**: release the file.

Real-life analogy: a bank counter. You walk in (connect), the teller takes your slip (cursor and execute), the manager stamps it so it becomes official (commit), and you leave (close). Until the stamp, the transaction is not final.

2.2 Your first program

```
import sqlite3

# 1. Connect. If school.db does not exist, SQLite creates it.
conn = sqlite3.connect("school.db")

# 2. Cursor
cur = conn.cursor()

# 3. Execute SQL
cur.execute("""
    CREATE TABLE IF NOT EXISTS students (
        student_id INTEGER PRIMARY KEY,
        name       TEXT NOT NULL,
        age        INTEGER
    )
""")

# 4. Commit (save)
conn.commit()

# 5. Close
conn.close()

print("Database ready!")
```

After running, a file called `school.db` appears in the same folder as the script. Show students this file in their file explorer. That is the whole database.

2.3 What each object does

Object	Created by	Job
Connection (`conn`)	`sqlite3.connect(path)`	Represents the open database. Controls `commit()`, `rollback()` and `close()`.

Object	Created by	Job
Cursor (`cur`)	`conn.cursor()`	Runs SQL with `execute()` and holds the results to read with `fetchone()` and `fetchall()`.

Shortcut: `conn.execute(sql)` creates a temporary cursor for you. It is fine for short scripts. In longer programs, an explicit cursor makes the code clearer.

2.4 Where is the database file?

`sqlite3.connect("school.db")` uses a **relative path**, meaning the file is created in the folder the script is **run from** (the current working directory), which is not always the script's folder. A very common student confusion is "I ran it again and my data disappeared" when in fact a second file was created somewhere else.

Safer pattern:

```
from pathlib import Path
import sqlite3

BASE_DIR = Path(__file__).parent          # folder containing this script
DB_PATH = BASE_DIR / "school.db"
conn = sqlite3.connect(DB_PATH)
```

2.5 In-memory database (great for practice)

```
conn = sqlite3.connect(":memory:")
```

This creates a database in RAM that vanishes when the program ends. Use it for experiments, exercises and tests, because nothing is left on disk and students cannot "break" anything permanently.

2.6 Using `with` for automatic commit or rollback

A connection works as a context manager, but be careful: **`with conn:` commits or rolls back, it does not close the connection.**

```
import sqlite3

conn = sqlite3.connect("school.db")

with conn:                                  # commit if no error, rollback if
error
    conn.execute("INSERT INTO students (name, age) VALUES (?, ?)", ("Aisha",
17))

conn.close()                                # you still must close it
yourself
```

To have everything closed automatically, combine it with `contextlib.closing`:

```
from contextlib import closing
import sqlite3

with closing(sqlite3.connect("school.db")) as conn:   # closes at the end
    with conn:                                        # commits or rolls back
        conn.execute("INSERT INTO students (name, age) VALUES (?, ?)",
("Brian", 18))
```

2.7 Understanding commit (the most misunderstood idea)

When you run INSERT, UPDATE or DELETE, the change first lives in a temporary "open transaction". It is **only visible to your connection** and is **not saved to the file** until you call `conn.commit()`. If your program ends without a commit, the changes are thrown away.

Real-life story: a shop attendant types a sale into a till but never presses "Confirm". The power goes off. No sale was recorded. That is what a missing `commit()` does.

```
conn.execute("INSERT INTO students (name, age) VALUES ('Carol', 19)")
# program ends here with no commit -> Carol is NOT saved
```

Note for newer Python: from Python 3.12 there is also an `autocommit` setting on `connect()` that changes this behaviour. For teaching, keep the default (manual `commit()` or `with conn:`), because it shows students exactly when data becomes permanent.

2.8 Common mistakes in this part

- Forgetting `commit()` and wondering why data is missing.

- Forgetting that `with conn:` does not close the connection.
- Running the script from a different folder and creating a second, empty database.
- Leaving the DB open in a viewer app (DB Browser for SQLite) while the script tries to write, which gives a "database is locked" error.

Part 3: Creating tables, data types and constraints

3.1 Designing before coding

Before typing `CREATE TABLE`, ask: *what things am I storing, and what do I need to know about each?* **Real-life story:** a school needs to track students. For each student: a unique ID, full name, date of birth, email, phone, and the date they joined. Each of those becomes a column.

3.2 CREATE TABLE syntax

```
CREATE TABLE table_name (
    column1  TYPE  constraints,
    column2  TYPE  constraints,
    ...
);
```

In Python, use a triple-quoted string so the SQL can span many lines:

```
cur.execute("""
    CREATE TABLE IF NOT EXISTS students (
        student_id  INTEGER PRIMARY KEY AUTOINCREMENT,
        full_name   TEXT    NOT NULL,
        email       TEXT    UNIQUE,
        phone       TEXT,
        birth_date  TEXT,
        joined_on   TEXT    DEFAULT CURRENT_DATE,
        is_active   INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1))
    )
""")
```

`IF NOT EXISTS` makes the command safe to run again. Without it, running twice raises `sqlite3.OperationalError: table students already exists`.

3.3 SQLite data types

SQLite is unusual: it uses **type affinity**, which means a column has a preferred type but SQLite is flexible about what you store. Teach the five storage classes first:

Storage class	Holds	Python type it maps to	Example
`NULL`	Missing or unknown value	`None`	A student with no phone
`INTEGER`	Whole numbers	`int`	age, quantity, ID
`REAL`	Decimal numbers	`float`	weight in kg, rating
`TEXT`	Text	`str`	name, email, date as text
`BLOB`	Raw bytes	`bytes`	a photo, a small file

What about dates and true/false? SQLite has no separate date or boolean type. By convention:

- Store dates and times as **TEXT in ISO format**: `"2026-11-04"` or `"2026-11-04 13:51:00"`. This sorts correctly alphabetically and works with SQLite's date functions.
- Store true/false as **INTEGER 0 or 1**.
- Store money as **INTEGER in the smallest unit** (for UGX, whole shillings are fine) to avoid decimal rounding errors. If you need cents, store cents as integers.

Teaching warning: SQLite will not stop you from putting text into an INTEGER column unless it cannot be converted. Add `CHECK` constraints (below) when the rule matters. You can also add `STRICT` to a table (SQLite 3.37 and newer) to make types enforced:

```
CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    name       TEXT NOT NULL,
    price_ugx  INTEGER NOT NULL
) STRICT;
```

3.4 Constraints: rules that protect your data

Constraints are rules the database enforces automatically, so bad data is rejected even if your Python code has a bug.

Constraint	What it does	Real-life meaning
`PRIMARY KEY`	Unique, never empty row ID	National ID number or student number
`AUTOINCREMENT`	Never re-uses an old ID	Receipt numbers that only go up
`NOT NULL`	The value is required	A student must have a name
`UNIQUE`	No two rows share this value	Two students cannot share one email
`DEFAULT value`	Used when you give no value	Joined date defaults to today
`CHECK (condition)`	Value must satisfy a rule	Age must be at least 5; price cannot be negative
`FOREIGN KEY`	Must match a row in another table	An enrolment must belong to a real student (Part 7)

Note on `INTEGER PRIMARY KEY`: in SQLite this alone already auto-numbers new rows (1, 2, 3…). `AUTOINCREMENT` adds the extra guarantee that IDs are never re-used even after deletes, with slightly more overhead. For most classroom work, either is fine.

3.5 A complete example: shop products with rules

```
cur.execute("""
    CREATE TABLE IF NOT EXISTS products (
        product_id  INTEGER PRIMARY KEY,
        name        TEXT    NOT NULL UNIQUE,
        category    TEXT    NOT NULL DEFAULT 'general',
        price_ugx   INTEGER NOT NULL CHECK (price_ugx >= 0),
        stock_qty   INTEGER NOT NULL DEFAULT 0 CHECK (stock_qty >= 0)
    )
""")
```

What the rules do:

- You cannot add a product without a name or price.
- Two products cannot have the same name.
- A negative price or negative stock is rejected with `sqlite3.IntegrityError`.

3.6 Changing and removing tables

```
# Add a column to an existing table
cur.execute("ALTER TABLE students ADD COLUMN address TEXT")

# Rename a table
cur.execute("ALTER TABLE students RENAME TO learners")

# Rename a column (SQLite 3.25+)
cur.execute("ALTER TABLE learners RENAME COLUMN full_name TO name")

# Delete a table completely (DANGEROUS: all its data is lost)
cur.execute("DROP TABLE IF EXISTS old_students")
```

Limitation to mention: older versions of SQLite cannot drop or change a column's type directly. The usual approach is to create a new table, copy the data across, drop the old table and rename the new one.

3.7 Looking at the structure (inspecting a database)

Students often want to "see what is inside". These queries help:

```
# List all tables in the database
cur.execute("SELECT name FROM sqlite_master WHERE type = 'table'")
print(cur.fetchall())

# See the columns of one table
cur.execute("PRAGMA table_info(students)")
for col in cur.fetchall():
    print(col)   # (cid, name, type, notnull, default_value, is_pk)
```

3.8 Common mistakes in this part

- Putting a trailing comma after the last column, which causes a syntax error.
- Using `DROP TABLE` without thinking, which permanently deletes all the rows.

- Storing dates as `15/03/2026`, which will not sort or compare correctly. Always use `2026-03-15`.
- Storing money as `REAL`, which can produce results like 0.30000000000000004. Prefer whole integers.

Part 4: Inserting data (Create)

4.1 Basic INSERT

```
INSERT INTO table_name (col1, col2, col3) VALUES (value1, value2, value3);
```

Real-life story: a new student, Aisha Nakato, registers at the school front desk. The clerk types her details into the system. That is one INSERT.

4.2 The right way: placeholders (`?`)

```
cur.execute(
    "INSERT INTO students (full_name, email, phone) VALUES (?, ?, ?)",
    ("Aisha Nakato", "aisha@example.com", "0771234567")
)
conn.commit()

print("New student ID:", cur.lastrowid)   # the auto-generated primary key
```

Rules to teach:

- Put a `?` where each value goes. Never type the value into the SQL string.
- Pass the values as a **tuple** in the same order. For one value, you need a trailing comma: `("Aisha",)`. This is a very common error.
- `cur.lastrowid` gives you the ID of the row you just inserted. Real systems use it immediately to create linked rows (for example, the student's first enrolment).

SQLite also supports named placeholders, which are clearer when there are many values:

```
cur.execute(
    "INSERT INTO students (full_name, email, phone) VALUES (:name, :email,
:phone)",
    {"name": "Brian Okello", "email": "brian@example.com", "phone":
"0759876543"}
)
conn.commit()
```

4.3 SQL injection: why placeholders are not optional

This is the single most important security lesson. **Never build SQL by joining strings with user input.**

Dangerous code (do NOT do this):

```
name = input("Student name: ")
cur.execute(f"SELECT * FROM students WHERE full_name = '{name}'")   # BAD
```

If a user types `x' OR '1'='1`, the query becomes:

```
SELECT * FROM students WHERE full_name = 'x' OR '1'='1'
```

That condition is always true, so the attacker sees **every student**. In other attacks, input can delete tables or steal passwords. Websites and apps have lost millions of records this way.

Safe code:

```
name = input("Student name: ")
cur.execute("SELECT * FROM students WHERE full_name = ?", (name,))   # GOOD
```

With a placeholder, the input is always treated as plain data and can never change the meaning of the command. A live demo of the bad version (on an in-memory database) is the best way to make this lesson stick.

Limit to remember: placeholders work for **values** only. You cannot use `?` for a table name or column name. If you must choose a column dynamically, check the name against a fixed allowed list in Python first:

```
ALLOWED_SORT = {"full_name", "joined_on", "email"}
sort_col = "joined_on"
if sort_col not in ALLOWED_SORT:
    raise ValueError("Invalid sort column")
cur.execute(f"SELECT * FROM students ORDER BY {sort_col}")
```

4.4 Inserting many rows: `executemany`

Real-life story: at the start of term, 200 students are uploaded from a spreadsheet. Running `execute` in a loop works but is slower and clumsier.

```
new_students = [
    ("Carol Namubiru", "carol@example.com", "0701111111"),
    ("David Ssemwogerere", "david@example.com", "0702222222"),
    ("Esther Auma", "esther@example.com", "0703333333"),
]

cur.executemany(
    "INSERT INTO students (full_name, email, phone) VALUES (?, ?, ?)",
    new_students
)
conn.commit()
print("Rows inserted:", cur.rowcount)   # 3
```

If any one row breaks a rule (for example, a duplicate email), the whole `executemany` call raises an error. Part 8 shows how to make this all-or-nothing safely with transactions.

4.5 Inserting from a CSV file (very practical)

Schools and farms keep data in Excel or CSV. Importing it is a real task students will do.

```
import csv
import sqlite3

with open("students.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)          # first row = column names
    rows = [(r["full_name"], r["email"], r["phone"]) for r in reader]

conn = sqlite3.connect("school.db")
with conn:
    conn.executemany(
        "INSERT INTO students (full_name, email, phone) VALUES (?, ?, ?)",
        rows
    )
conn.close()
```

4.6 Handling duplicates: `INSERT OR IGNORE` and `INSERT OR REPLACE`

Statement	What happens if the row would break a UNIQUE or PRIMARY KEY rule
`INSERT`	Raises `sqlite3.IntegrityError`
`INSERT OR IGNORE`	Silently skips that row
`INSERT OR REPLACE`	Deletes the old row and inserts the new one (the old row's other data is lost)

```
cur.execute(
    "INSERT OR IGNORE INTO students (full_name, email) VALUES (?, ?)",
    ("Aisha Nakato", "aisha@example.com")   # email already exists -> skipped
)
```

A safer, more precise option for updating on duplicates is the "upsert" (`ON CONFLICT ... DO UPDATE`), covered in Part 9.

4.7 Catching errors on insert

```
import sqlite3

try:
    cur.execute(
        "INSERT INTO students (full_name, email) VALUES (?, ?)",
        ("Fake Duplicate", "aisha@example.com")
    )
    conn.commit()
except sqlite3.IntegrityError as e:
    print("Could not add student:", e)   # UNIQUE constraint failed:
students.email
    conn.rollback()
```

4.8 Common mistakes in this part

- Building SQL with f-strings or `+` and user input (SQL injection).
- Passing `("Aisha")` instead of `("Aisha",)` for a single value, which gives "Incorrect number of bindings".
- Passing a list of values when the number of `?` marks does not match the number of values.
- Forgetting `commit()` after inserting.
- Inserting values in a different order from the column list.

Part 5: Reading data (SELECT)

Reading is what databases do most. Teach this part slowly and let students experiment.

5.0 Practice data (set this up once for the lesson)

Use a small shop database so every example has real output. Students run this in a fresh file or `:memory:`.

```
import sqlite3

conn = sqlite3.connect(":memory:")
cur = conn.cursor()

cur.execute("""
```

```
    CREATE TABLE products (
        product_id INTEGER PRIMARY KEY,
        name       TEXT NOT NULL,
        category   TEXT NOT NULL,
        price_ugx  INTEGER NOT NULL,
        stock_qty  INTEGER NOT NULL
    )
""")

products = [
    ("Rice 5kg",        "Food",      22000, 40),
    ("Sugar 1kg",       "Food",       5500, 120),
    ("Cooking oil 3L",  "Food",      24000, 25),
    ("Bar soap",        "Household",  2500, 200),
    ("Detergent 1kg",   "Household",  9000, 60),
    ("Exercise book",   "Stationery", 1500, 300),
    ("Pen (blue)",      "Stationery",  500, 500),
    ("Phone charger",   "Electronics",15000, 18),
    ("Earphones",       "Electronics",12000, 0),
]
cur.executemany(
    "INSERT INTO products (name, category, price_ugx, stock_qty) VALUES (?,
?, ?, ?)",
    products
)
conn.commit()
```

5.1 The basic SELECT and the fetch methods

```
cur.execute("SELECT name, price_ugx FROM products")

rows = cur.fetchall()       # list of tuples
for row in rows:
    print(row)
# ('Rice 5kg', 22000)
# ('Sugar 1kg', 5500) ...
```

Method	Returns	Use when
`fetchone()`	The next single row, or `None` if there are no more	You expect exactly one result (look up by ID)

Method	Returns	Use when
`fetchmany(n)`	A list of up to `n` rows	Processing in batches
`fetchall()`	A list of all remaining rows	Result is small enough for memory
Looping over the cursor (`for row in cur:`)	One row at a time	Large results; saves memory

```
cur.execute("SELECT name FROM products WHERE product_id = ?", (1,))
row = cur.fetchone()
if row is None:
    print("Not found")
else:
    print(row[0])           # Rice 5kg

# Memory-friendly loop
for name, price in cur.execute("SELECT name, price_ugx FROM products"):
    print(f"{name:<18} UGX {price:>7,}")
```

`SELECT *` means "all columns". It is fine for learning but, in real programs, name the columns you need. It is clearer and keeps working if someone adds columns later.

5.2 Rows as dictionaries: `row_factory`

Tuples force you to remember that `row[3]` is the price. A `sqlite3.Row` lets you use column names, which makes code far more readable.

```
conn.row_factory = sqlite3.Row
cur = conn.cursor()

cur.execute("SELECT * FROM products WHERE category = ?", ("Food",))
for row in cur.fetchall():
    print(row["name"], row["price_ugx"])

# Convert one row to a plain dict (e.g., to send as JSON)
print(dict(row))
```

Set `row_factory` right after connecting, before creating cursors. Teach students to use this from here on.

5.3 Filtering with WHERE

Real-life story: the shop owner asks, "What is cheaper than UGX 10,000?"

```
cur.execute("SELECT name, price_ugx FROM products WHERE price_ugx < ?",
(10000,))
```

Operator	Meaning	Example
`=` `!=` (or `<>`)	Equal, not equal	`category = 'Food'`
`<` `>` `<=` `>=`	Comparisons	`price_ugx >= 5000`
`AND`, `OR`, `NOT`	Combine conditions	`category = 'Food' AND stock_qty < 30`
`BETWEEN a AND b`	Inclusive range	`price_ugx BETWEEN 2000 AND 10000`
`IN (...)`	Match any in a list	`category IN ('Food', 'Household')`
`LIKE`	Pattern match (`%` = any text, `_` = one character)	`name LIKE 'S%'`
`IS NULL` / `IS NOT NULL`	Missing or present value	`phone IS NULL`

Important: to test for a missing value, use `IS NULL`. Writing `= NULL` never matches anything.

```
# Out-of-stock products
cur.execute("SELECT name FROM products WHERE stock_qty = 0")

# Food that is low in stock
cur.execute("SELECT name, stock_qty FROM products WHERE category = 'Food' AND
stock_qty < 30")

# Names containing 'oil' anywhere (search box feature)
term = "oil"
cur.execute("SELECT name FROM products WHERE name LIKE ?", (f"%{term}%",))
```

Note on the last line: the `%` signs go inside the **value**, not in the SQL. `LIKE` in SQLite is case-insensitive for plain English letters.

5.4 Sorting with ORDER BY and limiting with LIMIT

```
# Cheapest first
cur.execute("SELECT name, price_ugx FROM products ORDER BY price_ugx ASC")

# Most expensive first
cur.execute("SELECT name, price_ugx FROM products ORDER BY price_ugx DESC")

# Top 3 most expensive
cur.execute("SELECT name, price_ugx FROM products ORDER BY price_ugx DESC
LIMIT 3")

# Sort by category, then by price inside each category
cur.execute("SELECT category, name, price_ugx FROM products ORDER BY
category, price_ugx DESC")

# Pagination: page 2 with 3 items per page
cur.execute("SELECT name FROM products ORDER BY name LIMIT 3 OFFSET 3")
```

Real-life uses: "top 10 customers", "latest 5 orders", search results page by page.

5.5 Removing duplicates with DISTINCT

```
cur.execute("SELECT DISTINCT category FROM products")   # each category once
```

5.6 Calculated columns and aliases

```
cur.execute("""
    SELECT name,
           price_ugx,
           stock_qty,
           price_ugx * stock_qty AS stock_value_ugx
    FROM products
    ORDER BY stock_value_ugx DESC
""")
```

`AS` gives a result column a friendly name. SQLite also has useful built-in functions such as `UPPER()`, `LOWER()`, `LENGTH()`, `ROUND()`, `ABS()`, `COALESCE(x, default)` and `IFNULL(x, default)`.

5.7 Aggregate functions: summarising data

Aggregates collapse many rows into one answer.

Function	Meaning
`COUNT(*)`	Number of rows
`COUNT(column)`	Number of rows where the column is not NULL
`SUM(column)`	Total
`AVG(column)`	Average
`MIN(column)` / `MAX(column)`	Smallest / largest

```
cur.execute("SELECT COUNT(*) FROM products")
print(cur.fetchone()[0])                      # 9 products

cur.execute("SELECT SUM(price_ugx * stock_qty) FROM products")
print(cur.fetchone()[0])                      # total value of stock

cur.execute("SELECT ROUND(AVG(price_ugx), 0), MIN(price_ugx), MAX(price_ugx)
FROM products")
print(tuple(cur.fetchone()))
```

5.8 GROUP BY and HAVING

Real-life story: the owner wants a report: "For each category, how many products do I have and what is the total stock value?"

```
cur.execute("""
    SELECT category,
           COUNT(*)                   AS product_count,
           SUM(price_ugx * stock_qty)  AS stock_value_ugx
    FROM products
    GROUP BY category
    ORDER BY stock_value_ugx DESC
""")
for row in cur.fetchall():
    print(dict(row))
```

`GROUP BY` splits the rows into groups (one per category) and runs the aggregate on each group. **`HAVING` filters groups, while `WHERE` filters individual rows before grouping.**

```
# Only categories whose total stock value exceeds UGX 1,000,000
cur.execute("""
    SELECT category, SUM(price_ugx * stock_qty) AS stock_value_ugx
    FROM products
    WHERE stock_qty > 0              -- row filter, applied first
    GROUP BY category
    HAVING SUM(price_ugx * stock_qty) > 1000000   -- group filter, applied
after
""")
```

5.9 The order SQL is read (teach with a diagram on the board)

You **write** a query in this order, but SQLite **thinks** about it in a different order. Knowing this explains most student confusion.

Written order	Logical order of work
`SELECT`	5th: choose columns
`FROM`	1st: pick the table
`WHERE`	2nd: filter rows
`GROUP BY`	3rd: make groups
`HAVING`	4th: filter groups
`ORDER BY`	6th: sort
`LIMIT`	7th: cut the list

5.10 CASE: if/else inside SQL

```
cur.execute("""
    SELECT name, stock_qty,
           CASE
               WHEN stock_qty = 0  THEN 'Out of stock'
               WHEN stock_qty < 30 THEN 'Low stock'
               ELSE 'OK'
           END AS stock_status
    FROM products
""")
```

5.11 Common mistakes in this part

- Using `WHERE` with an aggregate (`WHERE COUNT(*) > 2`). Use `HAVING` instead.
- Selecting a plain column that is neither grouped nor aggregated alongside `GROUP BY`.
- Writing `= NULL` instead of `IS NULL`.
- Calling `fetchall()` twice and getting an empty list the second time (a cursor is used up once read).
- Reading `fetchone()` result as if it were a list of rows, or forgetting it can be `None`.
- Putting `%` in the SQL string with a placeholder instead of inside the value.

Part 6: Updating and deleting data

These two commands change or destroy existing data, so teach them with one golden rule.

Golden rule: always write the WHERE first. `UPDATE` or `DELETE` without `WHERE` affects **every row in the table**.

6.1 UPDATE

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
```

Real-life stories

- The shop raises the price of rice.
- A student changes their phone number.
- A sale happens, so the stock must go down.

```
# 1. Change one product's price
cur.execute(
    "UPDATE products SET price_ugx = ? WHERE name = ?",
    (23500, "Rice 5kg")
)
print("Rows changed:", cur.rowcount)    # 1
conn.commit()

# 2. Reduce stock after selling 3 phone chargers (calculate inside SQL)
cur.execute(
    "UPDATE products SET stock_qty = stock_qty - ? WHERE name = ? AND
stock_qty >= ?",
    (3, "Phone charger", 3)
)
if cur.rowcount == 0:
    print("Sale failed: not enough stock")
conn.commit()

# 3. 10% price increase for one category
cur.execute(
    "UPDATE products SET price_ugx = CAST(price_ugx * 1.10 AS INTEGER) WHERE
category = ?",
    ("Stationery",)
)
conn.commit()
```

Why the second example is good practice: doing `stock_qty - 3` **inside SQL** is safer than reading the number into Python, subtracting and writing it back, because if two sales happen at the same moment, the second approach can lose one of them. The extra `stock_qty >= ?` check stops stock going negative.

`cur.rowcount` tells you how many rows were affected. Use it to detect "nothing matched" (for example, a wrong ID).

6.2 DELETE

```
DELETE FROM table_name WHERE condition;
```

Real-life stories: removing a product the shop no longer sells; deleting a cancelled booking; clearing test data.

```
# Delete one product by ID
cur.execute("DELETE FROM products WHERE product_id = ?", (9,))
print(cur.rowcount, "row(s) deleted")
conn.commit()

# Delete out-of-stock stationery
cur.execute("DELETE FROM products WHERE category = ? AND stock_qty = 0",
("Stationery",))
conn.commit()
```

6.3 A safe habit: preview with SELECT first

Before a risky UPDATE or DELETE, run a SELECT with the same WHERE and look at what would be affected.

```
condition = "category = 'Electronics' AND stock_qty = 0"

cur.execute(f"SELECT name FROM products WHERE {condition}")     # teacher-
written, not user input
print("Will delete:", [r[0] for r in cur.fetchall()])

if input("Type YES to continue: ") == "YES":
    cur.execute(f"DELETE FROM products WHERE {condition}")
    conn.commit()
```

6.4 Soft delete: the way real systems delete

In real businesses you rarely erase records. A deleted student still has past payments and results that must be kept for audits. Instead, add a flag column and hide the row.

```
# Table has: is_active INTEGER NOT NULL DEFAULT 1
cur.execute("UPDATE students SET is_active = 0 WHERE student_id = ?", (5,))
conn.commit()

# Normal screens only show active students
cur.execute("SELECT * FROM students WHERE is_active = 1")
```

Advantages: you can "undelete", you keep history, and linked records (payments, results) stay valid.

6.5 TRUNCATE does not exist in SQLite

To empty a table, use `DELETE FROM table_name` (with no WHERE, on purpose). To reset auto-numbering as well (rarely needed):

```
cur.execute("DELETE FROM products")
cur.execute("DELETE FROM sqlite_sequence WHERE name = 'products'")   # only
if the table uses AUTOINCREMENT
conn.commit()
```

6.6 Common mistakes in this part

- Running `UPDATE` or `DELETE` with no `WHERE` and changing every row. The best protection is a backup (Part 9) and doing risky work inside a transaction (Part 8).
- Forgetting `commit()`, then seeing the change in your program but not after restarting.
- Updating by `name` when names are not unique. Prefer the primary key (`WHERE product_id = ?`).
- Not checking `rowcount`, so a mistyped ID fails silently.

Part 7: Relationships, foreign keys and JOINs

This is the part that turns "a spreadsheet in a file" into a real database.

7.1 Why split data into several tables?

Real-life story: imagine one big sheet for the school with these columns: student name, student phone, course name, course fee, teacher. If Aisha takes 3 courses, her name and phone are typed 3 times. When she changes her phone number, you must fix 3 rows, and if you miss one, the data disagrees with itself. This is called **data duplication**, and it causes errors.

The fix: keep each fact in **one** place and link tables using IDs. This design practice is called **normalisation**.

7.2 Types of relationships

Relationship	Meaning	Example
One-to-many	One row in A relates to many rows in B	One student makes many payments
Many-to-many	Many rows in A relate to many rows in B	Students take many courses, and each course has many students
One-to-one	One row in A relates to exactly one in B	A student and their profile photo record (rare)

A many-to-many relationship is always built with a **linking table** (also called a junction or bridge table). Here, `enrolments` links `students` and `courses`.

7.3 The school schema

```
import sqlite3

conn = sqlite3.connect("school.db")
conn.row_factory = sqlite3.Row
conn.execute("PRAGMA foreign_keys = ON")     # IMPORTANT, see 7.4

conn.executescript("""
CREATE TABLE IF NOT EXISTS students (
    student_id INTEGER PRIMARY KEY,
    full_name  TEXT NOT NULL,
    email      TEXT UNIQUE,
    phone      TEXT,
    joined_on  TEXT NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE IF NOT EXISTS courses (
    course_id  INTEGER PRIMARY KEY,
    title      TEXT NOT NULL UNIQUE,
    fee_ugx    INTEGER NOT NULL CHECK (fee_ugx >= 0)
);

CREATE TABLE IF NOT EXISTS enrolments (
    enrolment_id INTEGER PRIMARY KEY,
    student_id   INTEGER NOT NULL,
```

```
    course_id    INTEGER NOT NULL,
    enrolled_on  TEXT NOT NULL DEFAULT CURRENT_DATE,
    UNIQUE (student_id, course_id),                      -- cannot enrol
twice
    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE
CASCADE,
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)   ON DELETE
RESTRICT
);

CREATE TABLE IF NOT EXISTS payments (
    payment_id   INTEGER PRIMARY KEY,
    enrolment_id INTEGER NOT NULL,
    amount_ugx   INTEGER NOT NULL CHECK (amount_ugx > 0),
    paid_on      TEXT NOT NULL DEFAULT CURRENT_DATE,
    method       TEXT NOT NULL DEFAULT 'cash',
    FOREIGN KEY (enrolment_id) REFERENCES enrolments(enrolment_id)
);
""")
conn.commit()
```

New tool: `executescript()` runs **several** SQL statements separated by semicolons in one call. It is convenient for creating a whole schema. It does not accept `?` placeholders, so use it for setup scripts only, not for user data. By default it also commits any open transaction before it runs, so do not call it in the middle of work you may want to undo.

7.4 Foreign keys must be switched on in SQLite

This surprises almost everyone. **SQLite does not enforce foreign keys by default.** You must run `PRAGMA foreign_keys = ON` **every time you connect** (it is a per-connection setting, not saved in the file).

With it on, the database refuses nonsense:

```
# There is no student with ID 999
try:
    conn.execute("INSERT INTO enrolments (student_id, course_id) VALUES (999,
1)")
except sqlite3.IntegrityError as e:
    print(e)       # FOREIGN KEY constraint failed
```

ON DELETE options (what to do with child rows when the parent is deleted):

Option	Behaviour	School meaning
`RESTRICT`	Block the delete	Cannot delete a course that still has enrolments
`CASCADE`	Delete the child rows too	Deleting a student also deletes their enrolments
`SET NULL`	Set the child column to NULL	Rare; the column must allow NULL

Tip: a helper function keeps this setting from being forgotten:

```
def get_connection(path="school.db"):
    conn = sqlite3.connect(path)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn
```

7.5 Sample data

```
conn.executemany("INSERT INTO students (full_name, email, phone) VALUES (?,
?, ?)", [
    ("Aisha Nakato",   "aisha@example.com", "0771234567"),
    ("Brian Okello",   "brian@example.com", "0759876543"),
    ("Carol Namubiru", "carol@example.com", "0701111111"),
    ("David Ssemwogerere", "david@example.com", None),
])
conn.executemany("INSERT INTO courses (title, fee_ugx) VALUES (?, ?)", [
    ("Python Basics", 300000),
    ("Web Development", 450000),
    ("Data Analysis",   400000),
])
conn.executemany("INSERT INTO enrolments (student_id, course_id) VALUES (?,
?)", [
    (1, 1), (1, 2),      # Aisha: Python Basics + Web Development
    (2, 1),              # Brian: Python Basics
    (3, 3),              # Carol: Data Analysis
])
conn.executemany("INSERT INTO payments (enrolment_id, amount_ugx, method)
VALUES (?, ?, ?)", [
    (1, 300000, "mobile money"),     # Aisha paid Python Basics in full
```

```
    (2, 200000, "mobile money"),     # Aisha paid half of Web Development
    (3, 150000, "cash"),             # Brian paid half of Python Basics
])
conn.commit()
```

Note: David has no enrolments, and Carol has no payments. These cases are useful for explaining the different JOIN types.

7.6 INNER JOIN: only the rows that match on both sides

Real-life question: "Show each enrolment with the student's name and the course title."

```
rows = conn.execute("""
    SELECT s.full_name, c.title, e.enrolled_on
    FROM enrolments AS e
    INNER JOIN students AS s ON s.student_id = e.student_id
    INNER JOIN courses  AS c ON c.course_id  = e.course_id
    ORDER BY s.full_name
""").fetchall()

for r in rows:
    print(r["full_name"], "->", r["title"])
# Aisha Nakato -> Python Basics
# Aisha Nakato -> Web Development
# Brian Okello -> Python Basics
# Carol Namubiru -> Data Analysis
```

Read it as: "take each enrolment, find the student with the matching ID, find the course with the matching ID, and put them side by side." David does not appear, because he has no enrolment. `AS s` and `AS e` are short **table aliases**, which keep joins readable.

7.7 LEFT JOIN: keep everything from the left table

Real-life question: "List ALL students, even those who have not enrolled yet, so the front desk can follow them up."

```
rows = conn.execute("""
    SELECT s.full_name, c.title
    FROM students AS s
    LEFT JOIN enrolments AS e ON e.student_id = s.student_id
    LEFT JOIN courses    AS c ON c.course_id  = e.course_id
    ORDER BY s.full_name
""").fetchall()

for r in rows:
    print(r["full_name"], "->", r["title"])      # David Ssemwogerere -> None
```

Where there is no match, the right-side columns are `NULL` (Python `None`). Combine this with `IS NULL` to find the unmatched rows only:

```
# Students who have NOT enrolled in anything
rows = conn.execute("""
    SELECT s.full_name, s.phone
    FROM students AS s
    LEFT JOIN enrolments AS e ON e.student_id = s.student_id
    WHERE e.enrolment_id IS NULL
""").fetchall()
```

7.8 Join types at a glance

Join	Returns	Supported in SQLite
`INNER JOIN` (or just `JOIN`)	Only rows with a match in both tables	Yes
`LEFT JOIN`	All left rows, plus matches from the right (or NULL)	Yes
`RIGHT JOIN`	All right rows, plus matches from the left	Yes (SQLite 3.39+); otherwise swap the table order and use LEFT
`FULL OUTER JOIN`	All rows from both sides	Yes (SQLite 3.39+)
`CROSS JOIN`	Every combination of rows	Yes (rarely needed)

7.9 Joins plus aggregates: real reports

Report 1: who owes fees? For each enrolment, compare the course fee with the total paid.

```
rows = conn.execute("""
    SELECT s.full_name,
           c.title,
           c.fee_ugx,
           COALESCE(SUM(p.amount_ugx), 0)              AS paid_ugx,
           c.fee_ugx - COALESCE(SUM(p.amount_ugx), 0)  AS balance_ugx
    FROM enrolments AS e
    JOIN students AS s ON s.student_id = e.student_id
    JOIN courses  AS c ON c.course_id  = e.course_id
    LEFT JOIN payments AS p ON p.enrolment_id = e.enrolment_id
    GROUP BY e.enrolment_id
    HAVING balance_ugx > 0
    ORDER BY balance_ugx DESC
""").fetchall()

for r in rows:
    print(f"{r['full_name']:<18}{r['title']:<18}owes UGX
{r['balance_ugx']:>9,}")
# Carol Namubiru    Data Analysis     owes UGX   400,000
# Aisha Nakato      Web Development   owes UGX   250,000
# Brian Okello      Python Basics     owes UGX   150,000
```

Explain why `COALESCE(..., 0)` is needed: for a student with no payments, `SUM` returns `NULL`, and `fee - NULL` is `NULL`. `COALESCE` swaps `NULL` for `0`.

Report 2: how many students per course?

```
rows = conn.execute("""
    SELECT c.title, COUNT(e.enrolment_id) AS students
    FROM courses AS c
    LEFT JOIN enrolments AS e ON e.course_id = c.course_id
    GROUP BY c.course_id
    ORDER BY students DESC
""").fetchall()
```

Report 3: income per month.

```
rows = conn.execute("""
    SELECT strftime('%Y-%m', paid_on) AS month, SUM(amount_ugx) AS income_ugx
    FROM payments
    GROUP BY month
    ORDER BY month
""").fetchall()
```

7.10 Common mistakes in this part

- Forgetting `PRAGMA foreign_keys = ON`, so bad links are silently accepted.
- Forgetting the `ON` condition in a join and getting a huge "every row with every row" result.
- Using `WHERE` on the right-hand table of a `LEFT JOIN` for a normal condition, which turns it back into an inner join. Put such conditions in the `ON` clause instead.
- Ambiguous column names: if both tables have `student_id`, write `s.student_id` or `e.student_id`.
- Joining in a way that counts rows twice, then summing a number that was repeated.

Part 8: Transactions and error handling

8.1 What is a transaction?

A transaction is a group of changes that must **all succeed or all be cancelled**. The database promises this using the ACID properties:

Letter	Property	Plain meaning
A	Atomic	All steps happen, or none do
C	Consistent	Rules (constraints) are never broken
I	Isolated	Other users do not see half-finished work
D	Durable	Once committed, data survives a power cut

Real-life story: mobile money transfer. Sending UGX 50,000 from Aisha to Brian has two steps: subtract from Aisha, add to Brian. If the power fails after step one, Aisha has lost money and Brian has not received it. A transaction makes the two steps one unbreakable unit.

8.2 Transfer example

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.execute("""
    CREATE TABLE wallets (
        owner   TEXT PRIMARY KEY,
        balance INTEGER NOT NULL CHECK (balance >= 0)
    )
""")
conn.executemany("INSERT INTO wallets VALUES (?, ?)", [("Aisha", 100000),
("Brian", 20000)])
conn.commit()

def transfer(conn, sender, receiver, amount):
    try:
        with conn:    # begins a transaction; commits on success, rolls back
on any error
            conn.execute("UPDATE wallets SET balance = balance - ? WHERE
owner = ?", (amount, sender))
            conn.execute("UPDATE wallets SET balance = balance + ? WHERE
owner = ?", (amount, receiver))
        print("Transfer complete")
    except sqlite3.IntegrityError:
        print("Transfer failed: insufficient funds. Nothing was changed.")

transfer(conn, "Aisha", "Brian", 50000)     # works
transfer(conn, "Aisha", "Brian", 900000)    # fails: CHECK (balance >= 0) is
violated, change is undone

print(conn.execute("SELECT * FROM wallets").fetchall())   # [('Aisha',
50000), ('Brian', 70000)]
```

In the failed transfer, the first UPDATE tried to push Aisha's balance below zero, the CHECK rule rejected it, and the `with conn:` block rolled everything back. Nothing was half-done.

8.3 Manual commit and rollback

```
try:
    cur.execute("INSERT INTO payments (enrolment_id, amount_ugx) VALUES (?,
?)", (2, 100000))
    cur.execute("UPDATE enrolments SET enrolled_on = ? WHERE enrolment_id =
?", ("2026-11-04", 2))
    conn.commit()          # make both permanent
except sqlite3.Error as e:
    conn.rollback()        # undo both
    print("Rolled back because of:", e)
```

Important idea: **before `commit()`, a rollback restores the database to how it was at the last commit.** It is the "undo" button for data.

8.4 When does a transaction begin?

With the default settings, Python's `sqlite3` automatically opens a transaction just before your first INSERT, UPDATE or DELETE, and keeps it open until you call `commit()` or `rollback()`. Plain SELECT statements do not open one. That is why forgetting `commit()` loses data.

8.5 The sqlite3 exception hierarchy

All database errors inherit from `sqlite3.Error`, so catching it handles everything, while the specific ones let you react differently.

Exception	Typical cause	Example message
`sqlite3.IntegrityError`	A constraint was broken (UNIQUE, NOT NULL, CHECK, FOREIGN KEY)	`UNIQUE constraint failed: students.email`
`sqlite3.OperationalError`	Problem running the SQL or using the database file: bad syntax, missing table, locked database	`no such table: student` or `database is locked`
`sqlite3.ProgrammingError`	Wrong use of the API	`Incorrect number of bindings supplied` or using a closed connection

Exception	Typical cause	Example message
`sqlite3.DatabaseError`	General database problem, for example the file is not a database	`file is not a database`
`sqlite3.Error`	The parent of all of the above	

8.6 A friendly error-handling pattern for real programs

```
def add_student(conn, full_name, email, phone=None):
    """Add a student. Returns the new ID, or None if it could not be
added."""
    try:
        with conn:
            cur = conn.execute(
                "INSERT INTO students (full_name, email, phone) VALUES (?, ?,
?)",
                (full_name, email, phone)
            )
            return cur.lastrowid
    except sqlite3.IntegrityError as e:
        if "UNIQUE" in str(e):
            print(f"A student with the email {email} already exists.")
        elif "NOT NULL" in str(e):
            print("Name is required.")
        else:
            print("Data rules were broken:", e)
        return None
    except sqlite3.OperationalError as e:
        print("Database problem:", e)
        return None
```

Teach students to show **friendly** messages to users and to log the technical message for the developer.

8.7 Database is locked

SQLite allows many readers but **one writer at a time**. If another program (or another open connection in your own program) is writing, you may see `database is locked`. Mitigations:

- Keep transactions short: do the work and commit quickly.
- Close connections when finished.
- Increase the wait time: `sqlite3.connect("school.db", timeout=10)` waits up to 10 seconds before giving up (the default is 5).
- Close DB Browser for SQLite (or any viewer) while your script writes.
- For apps with many simultaneous writers, consider enabling WAL mode (Part 9) or moving to PostgreSQL.

8.8 Common mistakes in this part

- Catching every error with a bare `except:` and hiding real bugs. Catch `sqlite3.Error` or the specific subclass.
- Doing a long slow task (like waiting for user input) in the middle of an open transaction, which blocks other writers.
- Assuming a rollback also reverses a `CREATE TABLE`. In SQLite, DDL changes can be rolled back too if inside a transaction, but beginners should test, not assume.
- Not realising that closing the connection without a `commit()` silently discards pending changes.

Part 9: Intermediate and advanced topics

These topics turn a student from "can use SQLite" into "can build real applications". Teach them after Parts 1 to 8 are solid.

9.1 Indexes: making searches fast

Real-life story: a textbook has an index at the back. To find "photosynthesis", you look it up in the index instead of reading every page. A database index does the same. Without one, searching 1,000,000 students for one email means checking every row. With an index, SQLite jumps straight to it.

```
conn.execute("CREATE INDEX IF NOT EXISTS idx_students_email ON
students(email)")
conn.execute("CREATE INDEX IF NOT EXISTS idx_payments_enrolment ON
payments(enrolment_id)")
```

Fact	Detail
Already indexed automatically	`PRIMARY KEY` and `UNIQUE` columns
Index the columns you often use in	`WHERE`, `JOIN ... ON`, `ORDER BY`
Cost	Slightly slower inserts and updates, and more disk space
Do not index	Tiny tables, or columns with very few different values (like a yes/no flag)

See whether SQLite uses an index:

```
for row in conn.execute("EXPLAIN QUERY PLAN SELECT * FROM students WHERE
email = ?", ("aisha@example.com",)):
    print(tuple(row))
# ... SEARCH students USING INDEX idx_students_email (email=?)   <- good
# ... SCAN students                                              <- reads
every row
```

Classroom demo that students love: insert 500,000 random rows into a table, time a search with `time.perf_counter()`, add an index, and time it again.

```
import sqlite3, time, random, string

conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE people (id INTEGER PRIMARY KEY, email TEXT)")
rows = [("".join(random.choices(string.ascii_lowercase, k=12)) +
"@mail.com",) for _ in range(500_000)]
conn.executemany("INSERT INTO people (email) VALUES (?)", rows)
target = rows[-1][0]

def timed():
    t = time.perf_counter()
    conn.execute("SELECT * FROM people WHERE email = ?",
(target,)).fetchone()
    return time.perf_counter() - t

print("Without index:", timed())
conn.execute("CREATE INDEX idx_email ON people(email)")
print("With index:   ", timed())
```

9.2 Views: saved queries

A view is a named SELECT that behaves like a read-only table. **Real-life story:** the school accountant runs the "who owes fees" report every morning. Save it once and reuse it.

```
conn.execute("""
    CREATE VIEW IF NOT EXISTS v_fee_balances AS
    SELECT e.enrolment_id,
           s.full_name,
           c.title,
           c.fee_ugx,
           COALESCE(SUM(p.amount_ugx), 0)              AS paid_ugx,
           c.fee_ugx - COALESCE(SUM(p.amount_ugx), 0)  AS balance_ugx
    FROM enrolments e
    JOIN students s ON s.student_id = e.student_id
    JOIN courses  c ON c.course_id  = e.course_id
    LEFT JOIN payments p ON p.enrolment_id = e.enrolment_id
    GROUP BY e.enrolment_id
""")

for r in conn.execute("SELECT * FROM v_fee_balances WHERE balance_ugx > 0
ORDER BY balance_ugx DESC"):
    print(dict(r))
```

Benefits: one place to fix the logic, simpler Python code, and you can hide complexity from less experienced team members. Remove with `DROP VIEW v_fee_balances`.

9.3 Subqueries: a query inside a query

Real-life question: "Which products cost more than the average price?"

```
conn.execute("""
    SELECT name, price_ugx
    FROM products
    WHERE price_ugx > (SELECT AVG(price_ugx) FROM products)
""")
```

```
# Students who have made at least one payment
conn.execute("""
    SELECT full_name FROM students
    WHERE student_id IN (
        SELECT e.student_id
        FROM enrolments e JOIN payments p ON p.enrolment_id = e.enrolment_id
    )
""")
```

`EXISTS` is a common alternative that answers "is there at least one matching row?":

```
conn.execute("""
    SELECT full_name FROM students s
    WHERE NOT EXISTS (SELECT 1 FROM enrolments e WHERE e.student_id =
s.student_id)
""")   # students with no enrolments
```

9.4 CTEs: `WITH` makes long queries readable

A Common Table Expression gives a name to an intermediate result so the main query reads top to bottom.

```
conn.execute("""
    WITH paid AS (
        SELECT enrolment_id, SUM(amount_ugx) AS paid_ugx
        FROM payments
        GROUP BY enrolment_id
    )
    SELECT s.full_name, c.title, c.fee_ugx, COALESCE(paid.paid_ugx, 0) AS
paid_ugx
    FROM enrolments e
    JOIN students s ON s.student_id = e.student_id
    JOIN courses  c ON c.course_id  = e.course_id
    LEFT JOIN paid  ON paid.enrolment_id = e.enrolment_id
""")
```

9.5 Combining result sets: UNION

```
# One mailing list from two tables (duplicates removed)
conn.execute("SELECT email FROM students UNION SELECT email FROM teachers")
# UNION ALL keeps duplicates and is faster
```

9.6 Window functions (SQLite 3.25 and newer)

Window functions calculate across related rows **without collapsing them** into one row the way `GROUP BY` does.

```
# Rank products by price inside each category
conn.execute("""
    SELECT category, name, price_ugx,
           RANK() OVER (PARTITION BY category ORDER BY price_ugx DESC) AS
price_rank
    FROM products
""")

# Running total of income by date
conn.execute("""
    SELECT paid_on, amount_ugx,
           SUM(amount_ugx) OVER (ORDER BY paid_on, payment_id) AS
running_total
    FROM payments
""")
```

Real uses: leaderboards, "top 3 per group", running balances, month-on-month comparisons.

9.7 Upsert: insert, or update if it already exists (SQLite 3.24+)

Real-life story: a farm scale sends daily weights. If a record for that plot and date already exists, update it, otherwise insert a new one.

```
conn.execute("""
    CREATE TABLE IF NOT EXISTS daily_totals (
        plot_id INTEGER,
        day     TEXT,
        kilos   REAL NOT NULL,
        PRIMARY KEY (plot_id, day)
    )
""")

conn.execute("""
    INSERT INTO daily_totals (plot_id, day, kilos) VALUES (?, ?, ?)
    ON CONFLICT (plot_id, day) DO UPDATE SET kilos = kilos + excluded.kilos
""", (1, "2026-11-04", 35.5))
conn.commit()
```

`excluded.kilos` means "the value I tried to insert". Running the statement twice for the same plot and day adds the weights together instead of failing or creating a duplicate.

9.8 RETURNING: get data back from a change (SQLite 3.35+)

```
row = conn.execute(
    "INSERT INTO students (full_name, email) VALUES (?, ?) RETURNING
student_id, joined_on",
    ("Faith Akello", "faith@example.com")
).fetchone()
conn.commit()
print(row["student_id"], row["joined_on"])
```

Also works with `UPDATE ... RETURNING` and `DELETE ... RETURNING`, for example to see exactly which rows were deleted.

9.9 Dates and times

SQLite stores dates as text, and offers built-in date functions that work on ISO-format text.

Expression	Result
`date('now')`	Today's date in UTC, e.g. `2026-11-04`
`datetime('now')`	Date and time in UTC

Expression	Result
`datetime('now', 'localtime')`	Date and time in the computer's local time zone
`date('now', '-7 days')`	The date 7 days ago
`date('now', 'start of month')`	First day of this month
`strftime('%Y-%m', paid_on)`	Year and month, for grouping reports
`julianday(b) - julianday(a)`	Number of days between two dates

Important for East Africa: `CURRENT_DATE`, `CURRENT_TIMESTAMP` and `datetime('now')` are in **UTC**, which is 3 hours behind Kampala. A payment recorded at 01:00 Kampala time on the 5th would be stamped as the 4th. Use `'localtime'` when the local date matters, or store the timestamp from Python.

```
# Payments from the last 30 days
conn.execute("SELECT * FROM payments WHERE paid_on >= date('now', '-30
days')")

# Days since each student joined
conn.execute("SELECT full_name, CAST(julianday('now') - julianday(joined_on)
AS INTEGER) AS days_enrolled FROM students")

# Passing a Python date as ISO text (works on every Python version)
from datetime import date, datetime
conn.execute("INSERT INTO payments (enrolment_id, amount_ugx, paid_on) VALUES
(?, ?, ?)",
             (1, 50000, date.today().isoformat()))

# Turning text back into a Python object
d = date.fromisoformat("2026-11-04")
t = datetime.fromisoformat("2026-11-04 13:51:00")
```

A note on adapters and converters: `sqlite3` can automatically convert Python `date` and `datetime` objects with `detect_types`, but the built-in default adapters were **deprecated in Python 3.12**. Teach the simple, reliable way instead: store ISO text, and call `isoformat()` and `fromisoformat()` yourself.

9.10 Triggers: automatic actions

A trigger runs SQL automatically when a table changes. **Real-life story:** every time a sale is recorded, the shop's stock should reduce itself, so no one can forget.

```
conn.executescript("""
CREATE TABLE IF NOT EXISTS sales (
    sale_id    INTEGER PRIMARY KEY,
    product_id INTEGER NOT NULL REFERENCES products(product_id),
    qty        INTEGER NOT NULL CHECK (qty > 0),
    sold_at    TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER IF NOT EXISTS trg_reduce_stock
AFTER INSERT ON sales
BEGIN
    UPDATE products SET stock_qty = stock_qty - NEW.qty WHERE product_id =
NEW.product_id;
END;
""")
```

`NEW` refers to the row just inserted (and `OLD` to the row before an update or delete). Triggers are also used for audit logs ("who changed this price, and when?") and for automatically setting an `updated_at` column.

Caution to teach: triggers work invisibly, so overusing them makes programs hard to debug. Document each one.

9.11 Your own SQL functions from Python

You can register a Python function and then call it inside SQL.

```
def shs(amount):
    return f"UGX {amount:,}" if amount is not None else ""

conn.create_function("fmt_ugx", 1, shs)

for r in conn.execute("SELECT name, fmt_ugx(price_ugx) AS price FROM
products"):
    print(r["name"], r["price"])
```

9.12 Storing files (BLOBs)

Small files such as a student photo or a signed form can be stored directly in the database as bytes.

```
with open("aisha.jpg", "rb") as f:
    photo_bytes = f.read()

conn.execute("ALTER TABLE students ADD COLUMN photo BLOB")
conn.execute("UPDATE students SET photo = ? WHERE student_id = ?",
(photo_bytes, 1))
conn.commit()

row = conn.execute("SELECT photo FROM students WHERE student_id =
1").fetchone()
with open("aisha_copy.jpg", "wb") as f:
    f.write(row["photo"])
```

Guidance: for large files (videos, many big images) store the file on disk and save only its **path** in the database. This keeps the database small and fast.

9.13 Storing flexible data as JSON

SQLite has JSON functions (`json_extract` , `json_each` , and more) in current builds. Combine them with Python's `json` module for fields whose shape varies.

```
import json

conn.execute("CREATE TABLE IF NOT EXISTS survey (id INTEGER PRIMARY KEY,
answers TEXT)")
conn.execute("INSERT INTO survey (answers) VALUES (?)",
             (json.dumps({"q1": "yes", "q2": 4, "tags": ["farm",
"coffee"]}),))
conn.commit()

print(conn.execute("SELECT json_extract(answers, '$.q2') FROM
survey").fetchone()[0])   # 4
```

Use JSON sparingly. If you often filter or join on a value, give it its own column.

9.14 Performance and safety settings (PRAGMA)

PRAGMA	Purpose
`PRAGMA foreign_keys = ON`	Enforce relationships (every connection)
`PRAGMA journal_mode = WAL`	Write-Ahead Logging: readers and a writer no longer block each other; recommended for apps with several users. The setting is saved in the file.
`PRAGMA synchronous = NORMAL`	A good speed and safety balance when used with WAL
`PRAGMA table_info(t)`	List a table's columns
`PRAGMA index_list(t)`	List a table's indexes
`PRAGMA integrity_check`	Check the database file for corruption

```
conn.execute("PRAGMA journal_mode = WAL")
print(conn.execute("PRAGMA integrity_check").fetchone()[0])    # 'ok'
```

Speed tip: inserting many rows is dramatically faster inside **one** transaction than committing after each row. Use `executemany` followed by a single `commit()`.

9.15 Backing up a database

Real-life rule: data you have not backed up is data you will eventually lose. Because SQLite is one file, copying the file works only when no one is writing. The safe built-in method is `Connection.backup()`:

```
import sqlite3
from datetime import datetime

src = sqlite3.connect("school.db")
stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
dst = sqlite3.connect(f"school_backup_{stamp}.db")

with dst:
    src.backup(dst)           # consistent copy, even while the app is
running

dst.close()
src.close()
```

Export a plain-text copy of the whole database (schema and data as SQL commands) with `iterdump()`:

```
with open("school_dump.sql", "w", encoding="utf-8") as f:
    for line in conn.iterdump():
        f.write(line + "\n")
```

9.16 Exporting query results to CSV (for Excel)

```
import csv

cur = conn.execute("SELECT * FROM v_fee_balances")
with open("fee_balances.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow([col[0] for col in cur.description])   # header row from
the column names
    writer.writerows(cur.fetchall())
```

`cur.description` holds the column names of the last query. This is how you make "Download report" buttons.

9.17 Attaching a second database

```
conn.execute("ATTACH DATABASE 'archive_2025.db' AS archive")
conn.execute("INSERT INTO archive.students SELECT * FROM students WHERE
joined_on < '2026-01-01'")
conn.commit()
conn.execute("DETACH DATABASE archive")
```

Useful for archiving old records into a separate file.

9.18 Using pandas with SQLite (for data analysis students)

```
import pandas as pd

df = pd.read_sql_query("SELECT * FROM v_fee_balances", conn)
print(df.groupby("title")["balance_ugx"].sum())

df.to_sql("fee_snapshot", conn, if_exists="replace", index=False)   #
DataFrame -> table
```

This requires installing pandas (`pip install pandas`). It is a natural bridge to data analysis lessons.

Part 10: Full project, School Management System

This capstone ties everything together. It is a text-menu program that a small school or training academy could really use: register students, manage courses, enrol, record fee payments, see who owes money, print a student statement, export to CSV and back up the data.

10.1 What each feature teaches

Feature	Concepts used
Register and search students	INSERT, placeholders, LIKE, UNIQUE, error handling
Courses and enrolment	Foreign keys, many-to-many linking table, `IntegrityError`
Record payments	Transactions, validation with a JOIN and SUM

Feature	Concepts used
Fee balances report	JOIN, LEFT JOIN, GROUP BY, HAVING, COALESCE
Student statement	Joins, parameterised queries
Monthly income	`strftime`, GROUP BY
Deactivate student	Soft delete
Export and backup	CSV, `cursor.description`, `Connection.backup()`

10.2 The complete program

Save as `school_manager.py` and run `python school_manager.py`. Let students type it in sections, test each function in the Python shell, then add the menu last.

```
"""school_manager.py - a small school management system (Python + SQLite)."""
import csv
import sqlite3
from contextlib import closing
from datetime import datetime
from pathlib import Path

DB_PATH = Path(__file__).parent / "school.db"

SCHEMA = """
CREATE TABLE IF NOT EXISTS students (
    student_id INTEGER PRIMARY KEY,
    full_name  TEXT NOT NULL,
    email      TEXT UNIQUE,
    phone      TEXT,
    joined_on  TEXT NOT NULL DEFAULT (date('now', 'localtime')),
    is_active  INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1))
);

CREATE TABLE IF NOT EXISTS courses (
    course_id INTEGER PRIMARY KEY,
    title     TEXT NOT NULL UNIQUE,
    fee_ugx   INTEGER NOT NULL CHECK (fee_ugx >= 0)
);

CREATE TABLE IF NOT EXISTS enrolments (
```

```
    enrolment_id INTEGER PRIMARY KEY,
    student_id   INTEGER NOT NULL,
    course_id    INTEGER NOT NULL,
    enrolled_on  TEXT NOT NULL DEFAULT (date('now', 'localtime')),
    UNIQUE (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)
);

CREATE TABLE IF NOT EXISTS payments (
    payment_id   INTEGER PRIMARY KEY,
    enrolment_id INTEGER NOT NULL,
    amount_ugx   INTEGER NOT NULL CHECK (amount_ugx > 0),
    paid_on      TEXT NOT NULL DEFAULT (date('now', 'localtime')),
    method       TEXT NOT NULL DEFAULT 'cash',
    FOREIGN KEY (enrolment_id) REFERENCES enrolments(enrolment_id)
);

CREATE INDEX IF NOT EXISTS idx_enrolments_student ON enrolments(student_id);
CREATE INDEX IF NOT EXISTS idx_payments_enrolment ON payments(enrolment_id);

CREATE VIEW IF NOT EXISTS v_fee_balances AS
SELECT e.enrolment_id,
       s.student_id,
       s.full_name,
       s.phone,
       c.title,
       c.fee_ugx,
       COALESCE(SUM(p.amount_ugx), 0)              AS paid_ugx,
       c.fee_ugx - COALESCE(SUM(p.amount_ugx), 0)  AS balance_ugx
FROM enrolments e
JOIN students s ON s.student_id = e.student_id
JOIN courses  c ON c.course_id  = e.course_id
LEFT JOIN payments p ON p.enrolment_id = e.enrolment_id
GROUP BY e.enrolment_id;
"""

STARTER_COURSES = [
    ("Python Basics", 300000),
    ("Web Development", 450000),
    ("Data Analysis", 400000),
]
```

```
# ---------- connection and setup ----------

def get_connection(path=DB_PATH):
    conn = sqlite3.connect(path)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn


def init_db(conn):
    conn.executescript(SCHEMA)
    with conn:
        conn.executemany(
            "INSERT OR IGNORE INTO courses (title, fee_ugx) VALUES (?, ?)",
STARTER_COURSES
        )


# ---------- students ----------

def add_student(conn, full_name, email=None, phone=None):
    """Returns the new student_id, or None if it could not be added."""
    full_name = full_name.strip()
    if not full_name:
        print("Name is required.")
        return None
    email = email.strip().lower() if email else None    # empty email becomes
NULL
    try:
        with conn:
            cur = conn.execute(
                "INSERT INTO students (full_name, email, phone) VALUES (?, ?,
?)",
                (full_name, email, phone or None),
            )
            return cur.lastrowid
    except sqlite3.IntegrityError as e:
        print("Could not add student:", "that email is already used." if
"UNIQUE" in str(e) else e)
        return None
```

```
def find_students(conn, term):
    pattern = f"%{term}%"
    return conn.execute(
        """SELECT student_id, full_name, email, phone, joined_on
           FROM students
           WHERE is_active = 1
             AND (full_name LIKE ? OR email LIKE ? OR phone LIKE ?)
           ORDER BY full_name""",
        (pattern, pattern, pattern),
    ).fetchall()


def deactivate_student(conn, student_id):
    with conn:
        cur = conn.execute(
            "UPDATE students SET is_active = 0 WHERE student_id = ? AND
is_active = 1", (student_id,)
        )
    return cur.rowcount == 1


# ---------- courses and enrolment ----------

def list_courses(conn):
    return conn.execute(
        """SELECT c.course_id, c.title, c.fee_ugx, COUNT(e.enrolment_id) AS
students
           FROM courses c LEFT JOIN enrolments e ON e.course_id = c.course_id
           GROUP BY c.course_id ORDER BY c.title"""
    ).fetchall()


def add_course(conn, title, fee_ugx):
    try:
        with conn:
            cur = conn.execute("INSERT INTO courses (title, fee_ugx) VALUES
(?, ?)", (title.strip(), fee_ugx))
            return cur.lastrowid
    except sqlite3.IntegrityError as e:
        print("Could not add course:", e)
        return None
```

```
def enrol(conn, student_id, course_id):
    try:
        with conn:
            cur = conn.execute(
                "INSERT INTO enrolments (student_id, course_id) VALUES (?,
?)", (student_id, course_id)
            )
            return cur.lastrowid
    except sqlite3.IntegrityError as e:
        if "UNIQUE" in str(e):
            print("This student is already enrolled in that course.")
        else:
            print("Unknown student or course ID.")
        return None


# ---------- payments ----------

def get_balance(conn, enrolment_id):
    row = conn.execute(
        "SELECT balance_ugx FROM v_fee_balances WHERE enrolment_id = ?",
(enrolment_id,)
    ).fetchone()
    return None if row is None else row["balance_ugx"]


def record_payment(conn, enrolment_id, amount_ugx, method="cash"):
    balance = get_balance(conn, enrolment_id)
    if balance is None:
        print("No such enrolment.")
        return False
    if amount_ugx <= 0 or amount_ugx > balance:
        print(f"Amount must be between 1 and the balance of UGX
{balance:,}.")
        return False
    with conn:
        conn.execute(
            "INSERT INTO payments (enrolment_id, amount_ugx, method) VALUES
(?, ?, ?)",
            (enrolment_id, amount_ugx, method),
        )
    return True
```

```
# ---------- reports ----------

def outstanding_balances(conn):
    return conn.execute(
        "SELECT * FROM v_fee_balances WHERE balance_ugx > 0 ORDER BY
balance_ugx DESC"
    ).fetchall()


def student_statement(conn, student_id):
    return conn.execute(
        "SELECT enrolment_id, title, fee_ugx, paid_ugx, balance_ugx FROM
v_fee_balances WHERE student_id = ?",
        (student_id,),
    ).fetchall()


def monthly_income(conn):
    return conn.execute(
        """SELECT strftime('%Y-%m', paid_on) AS month, SUM(amount_ugx) AS
income_ugx
           FROM payments GROUP BY month ORDER BY month DESC"""
    ).fetchall()


def export_query_to_csv(conn, sql, filename):
    cur = conn.execute(sql)
    with open(filename, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow([col[0] for col in cur.description])
        writer.writerows(cur.fetchall())


def backup_database(conn):
    stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    target = DB_PATH.with_name(f"school_backup_{stamp}.db")
    with closing(sqlite3.connect(target)) as dst:
        conn.backup(dst)
    return target


# ---------- text menu ----------
```

```
MENU = """
===== SCHOOL MANAGER =====
1. Register student        6. Record payment
2. Search students         7. Outstanding fees report
3. List courses            8. Student statement
4. Add course              9. Monthly income
5. Enrol student in course 10. Export fees to CSV
11. Deactivate student     12. Backup database
0. Quit
"""


def ask_int(prompt):
    try:
        return int(input(prompt).strip().replace(",", ""))
    except ValueError:
        print("Please type a whole number.")
        return None


def show_rows(rows, columns):
    if not rows:
        print("(nothing found)")
    for r in rows:
        print(" | ".join(str(r[c]) for c in columns))


def main():
    with closing(get_connection()) as conn:
        init_db(conn)
        while True:
            print(MENU)
            choice = input("Choose: ").strip()

            if choice == "1":
                sid = add_student(conn, input("Full name: "), input("Email
(optional): "), input("Phone: "))
                if sid:
                    print("Student registered with ID", sid)
            elif choice == "2":
                show_rows(find_students(conn, input("Search (name, email or
phone): ")),
```

```
                          ["student_id", "full_name", "email", "phone"])
            elif choice == "3":
                for c in list_courses(conn):
                    print(f"{c['course_id']}. {c['title']:<18} UGX
{c['fee_ugx']:>9,}  ({c['students']} students)")
            elif choice == "4":
                fee = ask_int("Fee in UGX: ")
                if fee is not None:
                    add_course(conn, input("Course title: "), fee)
            elif choice == "5":
                sid, cid = ask_int("Student ID: "), ask_int("Course ID: ")
                if sid and cid and enrol(conn, sid, cid):
                    print("Enrolled.")
            elif choice == "6":
                eid, amount = ask_int("Enrolment ID: "), ask_int("Amount
(UGX): ")
                if eid and amount and record_payment(conn, eid, amount,
input("Method (cash/mobile money): ") or "cash"):
                    print("Payment recorded.")
            elif choice == "7":
                for r in outstanding_balances(conn):
                    print(f"{r['full_name']:<20}{r['title']:<18}owes UGX
{r['balance_ugx']:>9,}  ({r['phone'] or 'no phone'})")
            elif choice == "8":
                sid = ask_int("Student ID: ")
                for r in (student_statement(conn, sid) if sid else []):
                    print(f"[{r['enrolment_id']}] {r['title']:<18} fee
{r['fee_ugx']:>9,}  paid {r['paid_ugx']:>9,}  balance
{r['balance_ugx']:>9,}")
            elif choice == "9":
                for r in monthly_income(conn):
                    print(f"{r['month']}  UGX {r['income_ugx']:>11,}")
            elif choice == "10":
                export_query_to_csv(conn, "SELECT * FROM v_fee_balances",
"fee_balances.csv")
                print("Saved fee_balances.csv")
            elif choice == "11":
                sid = ask_int("Student ID: ")
                print("Done." if sid and deactivate_student(conn, sid) else
"No active student with that ID.")
            elif choice == "12":
                print("Backup saved to", backup_database(conn))
            elif choice == "0":
```

```
                print("Goodbye!")
                break
            else:
                print("Invalid choice.")


if __name__ == "__main__":
    main()
```

10.3 Design decisions to point out to students

- **Every function takes `conn` as a parameter** instead of using a global. This makes functions easy to test with `get_connection(":memory:")`.
- **SQL lives in functions; the menu only calls functions.** This is separation of concerns, and it is how real applications are organised.
- **The `v_fee_balances` view is used twice** (reports and payment validation), so the money logic exists in exactly one place.
- **Payments are validated** before they are written, so the program cannot record more than is owed.
- **Soft delete** (`is_active`) keeps financial history intact.
- **Empty email becomes `NULL`**, because `UNIQUE` allows many `NULL` values but would reject two empty strings.

10.4 Testing without touching real data

```
from school_manager import get_connection, init_db, add_student, enrol,
record_payment, get_balance

conn = get_connection(":memory:")      # throw-away database
init_db(conn)

sid = add_student(conn, "Test Student", "t@example.com", "0700000000")
eid = enrol(conn, sid, 1)               # Python Basics (300,000)
assert get_balance(conn, eid) == 300000

assert record_payment(conn, eid, 100000) is True
assert get_balance(conn, eid) == 200000

assert record_payment(conn, eid, 999999) is False   # more than owed
assert get_balance(conn, eid) == 200000             # unchanged
print("All tests passed")
```

10.5 Extension ideas (homework or a final project)

- Add a `teachers` table and a `teacher_id` on courses (one-to-many).
- Add cohorts or terms, so the same course can run several times a year.
- Add attendance (table of enrolment, date, present or absent) and a report of students who missed 3 or more sessions.
- Add exam scores and a class ranking using window functions.
- Add a receipt number and print a formatted receipt.
- Add a simple login with **hashed** passwords (use `hashlib` or a library such as `bcrypt`; never store plain passwords).
- Replace the text menu with a web interface (Flask) or a desktop interface (Tkinter), keeping the same database functions.

Part 11: Second project, Coffee Farm Harvest Tracker

This project is smaller and more report-focused. It shows students that SQLite is not only for schools and shops: a farmer can track harvests, sales and costs, and ask the questions that decide whether the farm makes money. All numbers below are invented sample data for teaching.

11.1 The questions the farmer wants answered

1. How many kilos did each plot produce, and how many kilos per tree?
2. Which months give the biggest harvests?
3. How much coffee is still in the store (harvested minus sold)?
4. How much money came in, how much went out, and what is the profit?
5. Where does the money go: labour, fertiliser, transport?
6. What does the cash book look like day by day, with a running balance?

11.2 Schema and sample data

```
import csv
import sqlite3

conn = sqlite3.connect("farm.db")
conn.row_factory = sqlite3.Row
conn.execute("PRAGMA foreign_keys = ON")

conn.executescript("""
CREATE TABLE IF NOT EXISTS plots (
    plot_id   INTEGER PRIMARY KEY,
    name      TEXT NOT NULL UNIQUE,
    trees     INTEGER NOT NULL CHECK (trees > 0),
    intercrop TEXT                      -- e.g. 'soya beans', or NULL if
none
);

CREATE TABLE IF NOT EXISTS harvests (
    harvest_id   INTEGER PRIMARY KEY,
    plot_id      INTEGER NOT NULL REFERENCES plots(plot_id),
    harvested_on TEXT NOT NULL,         -- ISO date: YYYY-MM-DD
    kilos        REAL NOT NULL CHECK (kilos > 0),
    notes        TEXT
);

CREATE TABLE IF NOT EXISTS sales (
    sale_id            INTEGER PRIMARY KEY,
    sold_on            TEXT NOT NULL,
    buyer              TEXT NOT NULL,
    kilos              REAL NOT NULL CHECK (kilos > 0),
    price_per_kilo_ugx INTEGER NOT NULL CHECK (price_per_kilo_ugx > 0)
```

```
);

CREATE TABLE IF NOT EXISTS expenses (
    expense_id  INTEGER PRIMARY KEY,
    spent_on    TEXT NOT NULL,
    category    TEXT NOT NULL,           -- labour, fertiliser, transport,
tools ...
    description TEXT,
    amount_ugx  INTEGER NOT NULL CHECK (amount_ugx > 0)
);
""")

with conn:
    conn.executemany("INSERT OR IGNORE INTO plots (name, trees, intercrop)
VALUES (?, ?, ?)", [
        ("Plot A", 200, "soya beans"),
        ("Plot B", 150, "soya beans"),
        ("Plot C", 100, None),
    ])
    conn.executemany("INSERT INTO harvests (plot_id, harvested_on, kilos)
VALUES (?, ?, ?)", [
        (1, "2026-05-12", 310.5), (1, "2026-06-03", 420.0), (1, "2026-10-20",
280.0),
        (2, "2026-05-14", 190.0), (2, "2026-06-05", 260.5), (2, "2026-10-22",
175.0),
        (3, "2026-06-08", 140.0), (3, "2026-10-25", 120.0),
    ])
    conn.executemany("INSERT INTO sales (sold_on, buyer, kilos,
price_per_kilo_ugx) VALUES (?, ?, ?, ?)", [
        ("2026-06-20", "Kasese Traders", 800.0, 6500),
        ("2026-10-30", "Masaka Coffee Co-op", 400.0, 7000),
    ])
    conn.executemany("INSERT INTO expenses (spent_on, category, description,
amount_ugx) VALUES (?, ?, ?, ?)", [
        ("2026-03-10", "fertiliser", "NPK, 10 bags", 750000),
        ("2026-05-10", "labour", "Picking, 6 workers x 2 days", 360000),
        ("2026-06-02", "labour", "Picking, 8 workers x 2 days", 480000),
        ("2026-06-18", "transport", "Truck to buyer", 200000),
        ("2026-08-15", "tools", "Pruning saws and tarps", 320000),
        ("2026-10-19", "labour", "Picking, 6 workers x 2 days", 360000),
    ])
```

Teaching notes: `INSERT OR IGNORE` on `plots` means the setup can be run twice without duplicating plots (the `UNIQUE` name triggers the ignore). The other tables would duplicate rows if re-run, so in a real project you would seed data only once, or use a check like "only if the table is empty".

11.3 A tiny helper to print any report

```
def show(title, sql, params=()):
    print(f"--- {title} ---")
    rows = conn.execute(sql, params).fetchall()
    for r in rows:
        print(dict(r))
    print()
    return rows
```

11.4 The reports

Report 1: yield per plot and per tree

```
show("Yield per plot", """
    SELECT p.name,
           p.trees,
           ROUND(COALESCE(SUM(h.kilos), 0), 1)              AS total_kilos,
           ROUND(COALESCE(SUM(h.kilos), 0) / p.trees, 2)    AS
kilos_per_tree
    FROM plots p
    LEFT JOIN harvests h ON h.plot_id = p.plot_id
    GROUP BY p.plot_id
    ORDER BY kilos_per_tree DESC
""")
```

Why this matters: Plot A may produce the most kilos only because it has the most trees. **Kilos per tree** is the fair comparison. A `LEFT JOIN` keeps a plot in the report even if it has no harvest yet.

Report 2: harvest by month

```
show("Harvest by month", """
    SELECT strftime('%Y-%m', harvested_on) AS month,
           ROUND(SUM(kilos), 1) AS kilos
    FROM harvests
    GROUP BY month
    ORDER BY month
""")
```

Report 3: coffee still in the store

```
show("Stock in store (kg)", """
    SELECT (SELECT COALESCE(SUM(kilos), 0) FROM harvests)
         - (SELECT COALESCE(SUM(kilos), 0) FROM sales)  AS kilos_in_store
""")
```

Two independent totals are calculated in subqueries and then subtracted. This avoids the "double counting" trap you get from joining harvests and sales together.

Report 4: profit

```
show("Profit", """
    SELECT (SELECT COALESCE(SUM(kilos * price_per_kilo_ugx), 0) FROM sales)
AS income_ugx,
           (SELECT COALESCE(SUM(amount_ugx), 0) FROM expenses)
AS costs_ugx,
           (SELECT COALESCE(SUM(kilos * price_per_kilo_ugx), 0) FROM sales)
         - (SELECT COALESCE(SUM(amount_ugx), 0) FROM expenses)
AS profit_ugx
""")
```

Report 5: where the money goes (with percentages, using a window function)

```
show("Expenses by category", """
    SELECT category,
           SUM(amount_ugx) AS total_ugx,
           ROUND(100.0 * SUM(amount_ugx) / SUM(SUM(amount_ugx)) OVER (), 1)
AS percent_of_costs
    FROM expenses
    GROUP BY category
    ORDER BY total_ugx DESC
""")
```

`SUM(SUM(amount_ugx)) OVER ()` means "the grand total of the group totals", so each category can be shown as a share of all costs.

Report 6: the cash book with a running balance

```
ledger_sql = """
    WITH ledger AS (
        SELECT sold_on AS day,
               'Sale to ' || buyer AS item,
               CAST(kilos * price_per_kilo_ugx AS INTEGER) AS amount_ugx
        FROM sales
        UNION ALL
        SELECT spent_on,
               category || ': ' || COALESCE(description, ''),
               -amount_ugx
        FROM expenses
    )
    SELECT day, item, amount_ugx,
           SUM(amount_ugx) OVER (ORDER BY day, item
                                 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS running_balance_ugx
    FROM ledger
    ORDER BY day, item
"""
show("Cash book", ledger_sql)
```

This combines two tables into one list of money in (positive) and money out (negative), then adds a running total. Farmers and accountants read exactly this kind of table.

11.5 Recording new data from Python functions

```
def record_harvest(plot_name, harvested_on, kilos, notes=None):
    row = conn.execute("SELECT plot_id FROM plots WHERE name = ?",
(plot_name,)).fetchone()
    if row is None:
        raise ValueError(f"Unknown plot: {plot_name}")
    with conn:
        conn.execute(
            "INSERT INTO harvests (plot_id, harvested_on, kilos, notes)
VALUES (?, ?, ?, ?)",
            (row["plot_id"], harvested_on, kilos, notes),
        )

record_harvest("Plot C", "2026-11-03", 95.5, "Late flush")
```

11.6 Export for the bank, buyers or a spreadsheet

```
cur = conn.execute(ledger_sql)
with open("cash_book.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow([c[0] for c in cur.description])
    writer.writerows(cur.fetchall())
```

11.7 Extension ideas

- Add a `workers` table and record days worked, then compute labour cost per kilo.
- Add a `rainfall` table and compare yield with seasonal rain.
- Add a `soya_harvests` table for the intercrop, and compare total income per plot (coffee plus soya).
- Add price history and a chart with `matplotlib`.
- Scale it to many farms: add a `farms` table and a `farm_id` column to every other table. This is the first step toward a multi-farm app.

Part 12: Best practices, common mistakes and debugging

12.1 The ten rules to make students repeat

1. **Always use `?` placeholders** for values. Never join user input into SQL strings.

2. **Commit deliberately.** Use `with conn:` so every change is committed or rolled back automatically.
3. **Close connections** (use `contextlib.closing`, or `conn.close()` in a `finally` block).
4. **Turn on foreign keys** with `PRAGMA foreign_keys = ON` for every connection.
5. **Let the database enforce rules** with `NOT NULL`, `UNIQUE`, `CHECK` and foreign keys. Do not rely on Python code alone.
6. **Write `WHERE` first** for UPDATE and DELETE, and check `rowcount`.
7. **Use the primary key** to identify rows, not names.
8. **Name your columns** in INSERT and SELECT instead of relying on `SELECT *` or column order.
9. **Store dates as ISO text** (`YYYY-MM-DD`) and money as whole integers.
10. **Back up** before risky work, and on a schedule for real data.

12.2 Error message decoder

Error message	What it means	Fix
`no such table: students`	Table does not exist, or you connected to a different (empty) database file	Print the DB path; check spelling; run the CREATE script
`no such column: nme`	Misspelt column name	Check the schema with `PRAGMA table_info(table)`
`table students already exists`	CREATE TABLE ran twice	Add `IF NOT EXISTS`
`UNIQUE constraint failed: students.email`	Duplicate value in a UNIQUE column	Check first, or catch `IntegrityError` and tell the user
`NOT NULL constraint failed: students.full_name`	A required value was missing or `None`	Validate input before inserting
`CHECK constraint failed`	Value broke a CHECK rule	Fix the value, or the rule if it is wrong

Error message	What it means	Fix
`FOREIGN KEY constraint failed`	Linked row does not exist, or you tried to delete a parent that has children	Insert the parent first, or change the delete order
`Incorrect number of bindings supplied. The current statement uses 2, and there are 1 supplied`	Count of `?` does not match the values, or a single value is not in a tuple	Use `(value,)` with the trailing comma
`database is locked`	Another connection is writing	Close other connections or viewers; keep transactions short; raise `timeout`
`near "FROM": syntax error`	A typo in the SQL, often a trailing comma or a missing comma	Print the SQL and read it slowly; test it in DB Browser
`Cannot operate on a closed database`	Using `conn` after `conn.close()`	Check the order of your code
`You can only execute one statement at a time`	Two statements in `execute()`	Use `executescript()` for setup, or separate calls
`sqlite3.ProgrammingError: Error binding parameter`	A Python type SQLite cannot store (list, dict, custom object)	Convert it to text, number or bytes (for example with `json.dumps`)

12.3 Debugging toolkit

- **Print the SQL and the parameters** before executing, when something looks wrong.
- Open the `.db` file in a viewer such as **DB Browser for SQLite** to see the real data (close it before running the script to avoid locks).
- Use `conn.set_trace_callback(print)` to print every SQL statement Python actually sends.
- Use `EXPLAIN QUERY PLAN` to see whether a query uses an index.
- Test queries in `:memory:` with a few known rows, so you can predict the result.
- Run `PRAGMA integrity_check` if you suspect a corrupt file.

```
conn.set_trace_callback(print)      # shows every statement as it runs
conn.execute("SELECT * FROM students WHERE student_id = ?", (1,))
```

12.4 Security checklist

- Parameterised queries everywhere (prevents SQL injection).
- Never store passwords in plain text. Store a salted hash.
- Limit what a feature can do. A "search" function should only run SELECT.
- Keep the database file out of public web folders.
- Restrict file permissions on the `.db` file on shared computers.
- Do not log personal data (phone numbers, national IDs) unnecessarily. Consider data protection duties when storing students' or customers' personal information.
- Back up to a second location, not only the same computer.

12.5 Performance checklist

- Wrap bulk changes in one transaction (`executemany` plus one `commit()`).
- Add indexes on columns used in `WHERE`, `JOIN` and `ORDER BY`.
- Select only the columns and rows you need.
- Use `LIMIT` for lists shown on screen.
- Use WAL mode if several processes access the database.
- Reuse one connection for a program's lifetime instead of connecting for every query.

12.6 Threads and web apps (a short warning)

A `sqlite3` connection by default can only be used in the **thread that created it**. Web frameworks such as Flask or Django often use several threads. The simple and safe approach is to create a **new connection per request** (and close it at the end). Avoid `check_same_thread=False` unless you add your own locking.

12.7 When SQLite is not the right tool

Choose PostgreSQL or MySQL when you need many users writing at the same moment, a database on a separate server shared by many apps, fine-grained user permissions, or very large data in the hundreds of gigabytes. The SQL you learned still applies. Python code changes only slightly (a different library and a different connect call).

Part 13: Exercises with solutions

Give students the questions first and keep the solutions for review. Exercises 3 to 9 use the `products` data from section 5.0. Exercises 11 and 12 use the school data from section 7.5.

Level 1: Connect, create, insert (Parts 2 to 4)

Exercise 1. Create a database `library.db` with a `books` table: an auto-numbered ID, a required title, a required author, a year, and a number of copies that defaults to 1 and can never be negative. Insert five books.

```
import sqlite3

conn = sqlite3.connect("library.db")
conn.execute("""
    CREATE TABLE IF NOT EXISTS books (
        book_id INTEGER PRIMARY KEY,
        title   TEXT NOT NULL,
        author  TEXT NOT NULL,
        year    INTEGER,
        copies  INTEGER NOT NULL DEFAULT 1 CHECK (copies >= 0)
    )
""")
with conn:
    conn.executemany("INSERT INTO books (title, author, year, copies) VALUES
(?, ?, ?, ?)", [
        ("Things Fall Apart", "Chinua Achebe", 1958, 3),
        ("The River Between", "Ngugi wa Thiong'o", 1965, 2),
        ("Kintu", "Jennifer Nansubuga Makumbi", 2014, 1),
        ("Song of Lawino", "Okot p'Bitek", 1966, 2),
        ("Half of a Yellow Sun", "Chimamanda Ngozi Adichie", 2006, 4),
    ])
conn.close()
```

Exercise 2. Write `add_book(conn, title, author, year=None, copies=1)` that returns the new ID, or prints a friendly message and returns `None` when a rule is broken (for example, `copies = -1`).

```
def add_book(conn, title, author, year=None, copies=1):
    try:
        with conn:
            cur = conn.execute(
                "INSERT INTO books (title, author, year, copies) VALUES (?,
?, ?, ?)",
                (title, author, year, copies),
            )
            return cur.lastrowid
    except sqlite3.IntegrityError as e:
        print("Could not add book:", e)
        return None
```

Level 2: Reading data (Part 5)

Exercise 3. List the name and price of products that cost between UGX 2,000 and UGX 15,000, cheapest first.

```
rows = conn.execute(
    "SELECT name, price_ugx FROM products WHERE price_ugx BETWEEN ? AND ?
ORDER BY price_ugx",
    (2000, 15000),
).fetchall()
```

Exercise 4. For each category, show the number of products and the average price rounded to a whole number.

```
rows = conn.execute("""
    SELECT category, COUNT(*) AS products, ROUND(AVG(price_ugx)) AS avg_price
    FROM products
    GROUP BY category
""").fetchall()
```

Exercise 5. Show only categories whose total stock value is above UGX 500,000.

```
rows = conn.execute("""
    SELECT category, SUM(price_ugx * stock_qty) AS stock_value
    FROM products
    GROUP BY category
    HAVING SUM(price_ugx * stock_qty) > 500000
""").fetchall()
```

Exercise 6. Write `search_products(conn, term)` that finds products whose name contains the search word, ignoring upper or lower case, and returns them sorted by name.

```
def search_products(conn, term):
    return conn.execute(
        "SELECT * FROM products WHERE name LIKE ? ORDER BY name", (f"%
{term}%",)
    ).fetchall()
```

Level 3: Changing data and transactions (Parts 6 and 8)

Exercise 7. Give a 5% discount to all Household products, rounded down to a whole shilling. Print how many were changed.

```
with conn:
    cur = conn.execute(
        "UPDATE products SET price_ugx = CAST(price_ugx * 0.95 AS INTEGER)
WHERE category = ?",
        ("Household",),
    )
print(cur.rowcount, "products discounted")
```

Exercise 8. Write `sell(conn, product_id, qty)` that reduces stock only if there is enough, and returns `True` or `False`.

```
def sell(conn, product_id, qty):
    with conn:
        cur = conn.execute(
            "UPDATE products SET stock_qty = stock_qty - ? WHERE product_id =
? AND stock_qty >= ?",
            (qty, product_id, qty),
        )
        return cur.rowcount == 1
```

Exercise 9. Use the `wallets` table from section 8.2. Prove that a failed transfer leaves both balances unchanged.

```
before = conn.execute("SELECT * FROM wallets ORDER BY owner").fetchall()
transfer(conn, "Brian", "Aisha", 10_000_000)       # too large, must fail
after = conn.execute("SELECT * FROM wallets ORDER BY owner").fetchall()
assert [tuple(r) for r in before] == [tuple(r) for r in after]
print("Rollback works")
```

Level 4: Relationships and reports (Parts 7 and 9)

Exercise 10. Extend the library with `members` and `loans` tables. Then list every book that is **currently on loan**, with the member's name, and a second list of **overdue** loans.

```
conn.executescript("""
CREATE TABLE IF NOT EXISTS members (
    member_id INTEGER PRIMARY KEY,
    full_name TEXT NOT NULL,
    phone     TEXT
);
CREATE TABLE IF NOT EXISTS loans (
    loan_id     INTEGER PRIMARY KEY,
    book_id     INTEGER NOT NULL REFERENCES books(book_id),
    member_id   INTEGER NOT NULL REFERENCES members(member_id),
    loaned_on   TEXT NOT NULL,
    due_on      TEXT NOT NULL,
    returned_on TEXT                     -- NULL means not yet returned
);
""")

on_loan = """
    SELECT b.title, m.full_name, l.loaned_on, l.due_on
    FROM loans l
    JOIN books   b ON b.book_id   = l.book_id
    JOIN members m ON m.member_id = l.member_id
    WHERE l.returned_on IS NULL
"""
overdue = on_loan + " AND l.due_on < date('now', 'localtime') ORDER BY
l.due_on"
```

Exercise 11. List students who have **not paid anything** at all (they may have no enrolment, or an enrolment with no payment).

```
rows = conn.execute("""
    SELECT s.full_name, s.phone
    FROM students s
    LEFT JOIN enrolments e ON e.student_id = s.student_id
    LEFT JOIN payments   p ON p.enrolment_id = e.enrolment_id
    GROUP BY s.student_id
    HAVING COUNT(p.payment_id) = 0
""").fetchall()
```

Exercise 12. Rank students by total money paid, highest first, using a window function.

```
rows = conn.execute("""
    SELECT full_name, total_paid, RANK() OVER (ORDER BY total_paid DESC) AS
position
    FROM (
        SELECT s.full_name, COALESCE(SUM(p.amount_ugx), 0) AS total_paid
        FROM students s
        LEFT JOIN enrolments e ON e.student_id = s.student_id
        LEFT JOIN payments   p ON p.enrolment_id = e.enrolment_id
        GROUP BY s.student_id
    )
    ORDER BY position
""").fetchall()
```

Exercise 13. Create an **audit trigger** that records every change of a product's price (old price, new price and time) in a `price_log` table.

```
conn.executescript("""
CREATE TABLE IF NOT EXISTS price_log (
    log_id     INTEGER PRIMARY KEY,
    product_id INTEGER NOT NULL,
    old_price  INTEGER NOT NULL,
    new_price  INTEGER NOT NULL,
    changed_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER IF NOT EXISTS trg_price_log
AFTER UPDATE OF price_ugx ON products
WHEN OLD.price_ugx <> NEW.price_ugx
BEGIN
    INSERT INTO price_log (product_id, old_price, new_price)
    VALUES (OLD.product_id, OLD.price_ugx, NEW.price_ugx);
END;
""")
```

Exercise 14. Show that an index makes a search faster. Use the timing demo from section 9.1, and use `EXPLAIN QUERY PLAN` to print the plan before and after creating the index. The first plan should say `SCAN`, and the second should say `SEARCH ... USING INDEX`.

Level 5: Mini-projects (pick one)

Project	Tables to design	Features to build
Library system	books, members, loans	Borrow, return, overdue list, fines
Shop till (POS)	products, sales, sale_items	Sell several items per receipt, stock reduces, daily sales report
Expense tracker	categories, expenses, budgets	Monthly spending by category, over-budget warning
Class attendance	students, classes, attendance	Mark attendance, percentage per student, absent 3 times alert
Voting or survey	questions, options, responses	Record answers, count results, prevent double voting
Boda boda or taxi log	riders, trips, fuel	Trips per day, income minus fuel, best day of week

Marking guide for mini-projects (out of 20):

Criterion	Marks
Sensible schema with primary and foreign keys and constraints	4
All CRUD operations work	4
Uses placeholders everywhere (no string-built SQL)	3
At least two JOIN or GROUP BY reports	4
Error handling and transactions where needed	3
Clean code, functions, and a short README	2

Part 14: Lesson plan, teaching tips and cheat sheet

14.1 Suggested plan: 8 sessions of about 2 hours

Each session follows the same rhythm: 10 minutes recap quiz, 30 minutes teaching with live coding, 50 minutes guided practice, 20 minutes challenge, 10 minutes homework briefing.

Session	Topic	Notes sections	Live demo	Homework
1	Databases, SQL and SQLite; connect, cursor, commit, close	Parts 1 and 2	Create `school.db`, show the file on disk; the "forgot commit" experiment	Create a database with one table and 5 rows
2	Tables, data types, constraints; INSERT with placeholders	Parts 3 and 4	Break each constraint on purpose; SQL injection demo on `:memory:`	Exercises 1 and 2

Session	Topic	Notes sections	Live demo	Homework
3	SELECT, WHERE, ORDER BY, LIMIT, LIKE, row_factory	Part 5 (5.0 to 5.6)	Build a product search	Exercises 3 and 6
4	Aggregates, GROUP BY, HAVING, CASE; UPDATE and DELETE	Part 5 (5.7 to 5.10) and Part 6	Shop stock report; soft delete	Exercises 4, 5, 7 and 8
5	Relationships, foreign keys, INNER and LEFT JOIN	Part 7	Draw the school schema on the board, then build it	Exercises 11 and 12
6	Transactions, errors; indexes, views, upsert, dates	Parts 8 and 9	Mobile money transfer with rollback; 500,000-row index timing demo	Exercises 9, 13 and 14
7	Build the School Manager (or Farm Tracker) together	Parts 10 and 11	Live-code functions one by one and test each in `:memory:`	Choose a mini-project and submit the schema design
8	Project presentations, review, backup and next steps	Parts 12 and 13	Each student demos their project and explains one design decision	Final project submission

If you teach shorter lessons, split sessions 4 and 6 in two, which gives a 10-session course.

14.2 Teaching tips that save time

- **Live-code, do not just show slides.** Make mistakes deliberately so students see error messages and learn to read them.
- **Use `:memory:` for demos**, so you can rerun from a clean state in seconds.
- **Install DB Browser for SQLite** (free) on every computer so students can see their tables as grids. It makes the abstract concrete.

- **Always draw the table diagram first** (boxes with column names and lines for relationships) before writing any CREATE TABLE.
- **Ask "what do you expect to see?"** before running each query. Prediction before execution builds understanding.
- **Teach SQL and Python separately at first.** Let students try queries in DB Browser, then wrap them in Python.
- **Use local examples** (UGX amounts, school fees, mobile money, coffee kilos, boda boda trips). Students remember them and see the use in their own lives.
- **Pair students**: one types, one reads the error aloud, then swap.
- **End every session with a working program**, even a tiny one.

14.3 Short quiz questions for recap (with answers)

Question	Answer
What is the difference between a connection and a cursor?	The connection is the open database (commit, rollback, close). The cursor runs SQL and holds results.
What happens if you insert and close without `commit()`?	The change is lost.
Why use `?` placeholders?	To prevent SQL injection and to handle quotes and types correctly.
What is a primary key?	A column that uniquely identifies each row.
What is the difference between WHERE and HAVING?	WHERE filters rows before grouping. HAVING filters groups after grouping.
Which join keeps students with no enrolments?	LEFT JOIN from students to enrolments.
Why must you run `PRAGMA foreign_keys = ON`?	SQLite does not enforce foreign keys unless you switch it on for each connection.
Does `with conn:` close the connection?	No. It only commits or rolls back.
How do you test for a missing value?	`IS NULL`, never `= NULL`.
Why store money as integers?	To avoid rounding errors from decimals.

14.4 One-page cheat sheet: Python side

Task	Code
Import	`import sqlite3`
Connect to a file	`conn = sqlite3.connect("data.db")`
Temporary database	`sqlite3.connect(":memory:")`
Columns by name	`conn.row_factory = sqlite3.Row`
Enforce relationships	`conn.execute("PRAGMA foreign_keys = ON")`
Run SQL	`cur = conn.execute(sql, params)`
Many rows	`conn.executemany(sql, list_of_tuples)`
Several statements	`conn.executescript(script)`
One row	`cur.fetchone()` (returns `None` if there are none)
Some rows	`cur.fetchmany(n)`
All rows	`cur.fetchall()`
Loop rows	`for row in cur:`
New row's ID	`cur.lastrowid`
Rows changed	`cur.rowcount`
Column names of last query	`[c[0] for c in cur.description]`
Save changes	`conn.commit()`
Undo changes	`conn.rollback()`
Auto commit or rollback	`with conn: ...`
Close	`conn.close()` (or `closing(...)`)
Backup	`src.backup(dst)`
Text dump	`conn.iterdump()`
Show every statement	`conn.set_trace_callback(print)`
Wait longer when locked	`sqlite3.connect(path, timeout=10)`

Task	Code
Catch errors	`except sqlite3.IntegrityError`, `sqlite3.OperationalError`, `sqlite3.Error`

14.5 One-page cheat sheet: SQL side

```
-- Structure
CREATE TABLE IF NOT EXISTS t (id INTEGER PRIMARY KEY, name TEXT NOT NULL, qty
INTEGER DEFAULT 0 CHECK (qty >= 0));
ALTER TABLE t ADD COLUMN note TEXT;
DROP TABLE IF EXISTS t;
CREATE INDEX IF NOT EXISTS idx_t_name ON t(name);
CREATE VIEW v AS SELECT ...;

-- Create
INSERT INTO t (name, qty) VALUES (?, ?);
INSERT OR IGNORE INTO t (...) VALUES (...);
INSERT INTO t (id, qty) VALUES (?, ?) ON CONFLICT (id) DO UPDATE SET qty =
excluded.qty;

-- Read
SELECT a, b FROM t WHERE x = ? AND y > ? ORDER BY a DESC LIMIT 10 OFFSET 20;
SELECT DISTINCT category FROM t;
SELECT category, COUNT(*), SUM(qty), AVG(qty) FROM t GROUP BY category HAVING
COUNT(*) > 2;
SELECT ... FROM a JOIN b ON b.a_id = a.id LEFT JOIN c ON c.b_id = b.id;
SELECT ... WHERE col IN (SELECT ...);
SELECT ... WHERE col LIKE '%text%' OR col IS NULL OR col BETWEEN 1 AND 5;
SELECT CASE WHEN qty = 0 THEN 'out' ELSE 'ok' END FROM t;
SELECT COALESCE(col, 0), ROUND(x, 2), strftime('%Y-%m', date_col) FROM t;
WITH name AS (SELECT ...) SELECT ... FROM name;
SELECT RANK() OVER (PARTITION BY g ORDER BY v DESC) FROM t;

-- Update and delete (WHERE first!)
UPDATE t SET qty = qty - ? WHERE id = ?;
DELETE FROM t WHERE id = ?;
```

```
-- Inspect
SELECT name FROM sqlite_master WHERE type = 'table';
PRAGMA table_info(t);
EXPLAIN QUERY PLAN SELECT ...;
```

14.6 Features and the SQLite version they need

Check your version with `sqlite3.sqlite_version`. Almost any current Python install includes a new enough SQLite for everything in these notes, but older computers may not.

Feature	Needs SQLite
Upsert (`ON CONFLICT DO UPDATE`)	3.24
Window functions (`OVER`)	3.25
`RENAME COLUMN`	3.25
`RETURNING`	3.35
`DROP COLUMN`	3.35
`STRICT` tables	3.37
`RIGHT JOIN` and `FULL OUTER JOIN`	3.39

14.7 Glossary

Term	Meaning
ACID	Atomic, Consistent, Isolated, Durable: the guarantees of a transaction
Alias	A temporary short name for a table or column (`AS`)
Atomic	All steps succeed or none do
Cascade	Automatically applying a delete to linked child rows
Constraint	A rule enforced by the database
CRUD	Create, Read, Update, Delete
Cursor	The object that runs SQL and holds results

Term	Meaning
Index	A lookup structure that speeds up searching
Normalisation	Organising tables so each fact is stored once
NULL	The marker for "missing or unknown" (not zero, not empty text)
ORM	Object-Relational Mapper (such as SQLAlchemy): a library that lets you work with tables as Python classes
Primary key	The unique ID column of a table
Foreign key	A column that refers to another table's primary key
Schema	The structure of the database
SQL injection	An attack where user input changes the meaning of a query
Transaction	A group of changes treated as one unit
View	A saved query that acts like a read-only table
WAL	Write-Ahead Logging, a journal mode that improves concurrent access

14.8 What to learn next

1. **SQLAlchemy** or the lighter **peewee**: work with tables as Python classes (an ORM).
2. **Flask** or **FastAPI**: put your database behind a web or mobile API.
3. **PostgreSQL**: the next step when a project outgrows SQLite.
4. **Database design in depth**: normal forms, many-to-many modelling, designing for reporting.
5. **pandas and charts**: analyse query results visually.
6. **Testing**: use `pytest` with an in-memory database fixture.